NEST 2026 Cohort Final Project

Secure Lab Build, Assessment and Hardening Report

Author: Michael Asante Anim

Email: animmichaelasante@gmail.com

School: University of Mines and Technology, Tarkwa

Date: October 11, 2026

Every account, hostname and data item in this report exists only inside an isolated virtual lab. The lab has no route to production networks, no real passwords or personal information are shown, and all testing was performed only against systems that I own.

Contents

1 Project Overview and Objectives **5**
1.1 Overview . . . 5
1.2 Objectives . . . 5
1.3 Summary of Work and Steps Followed . . . 6
1.4 Rules of Engagement . . . 6
1.5 Teamwork and Shared Resources . . . 6

2 Environment Design and Network Diagram **6**
2.1 Design Summary . . . 7
2.2 Machines and Roles . . . 7
2.3 Network Diagram . . . 8
2.4 How the Machines Communicate . . . 9

3 Installation and Configuration **9**
3.1 Virtualisation Platform . . . 9
3.2 Windows Domain Controller . . . 9
3.2.1 Organisational Unit, Users and Groups . . . 12
3.3 Windows Client and Domain Join Attempt . . . 14
3.3.1 Checks Before Joining the Domain . . . 15
3.3.2 Domain Join Attempt and Result . . . 15
3.4 Linux Web Server . . . 16
3.4.1 Network Configuration . . . 17
3.4.2 Web Server Software . . . 18
3.4.3 Homepage Deployment and Access Test . . . 19
3.5 Static IP Configuration on Kali . . . 20
3.6 Connectivity Test Between Kali and the Windows Client . . . 21
3.7 Verification That the Environment Works . . . 22

4 Assessment Scope, Methods and Findings **23**
4.1 Scope . . . 23
4.2 Tools Selected . . . 24
4.3 Methodology . . . 24
4.4 Summary of Findings . . . 25
4.5 Detailed Findings . . . 26
4.6 F1: Missing HTTP security headers on the web server . . . 26
4.7 F2: Anonymous SMB null session permitted on the domain controller . . . 29
4.8 F3: Host firewall on the Windows client blocks lateral-movement ports (informational) . . . 33
4.9 Limitations of the Assessment . . . 33

5 Log Evidence and Detection Observations **34**
5.1 Log Sources Reviewed . . . 34
5.2 Activity Correlation . . . 34
5.3 What Was Not Visible . . . 38

6 Recommendations, Implemented Improvement and Verification **38**
6.1 Prioritised Recommendations . . . 38
6.2 Implemented Improvement . . . 39
6.3 Verification . . . 39

7 Challenges, Limitations and Lessons Learned **43**
7.1 Challenges . . . 43
7.2 Limitations . . . 44
7.3 What I Learned . . . 45

8 References and Supporting Material **45**
8.1 Acknowledgements . . . 45
8.2 References . . . 46

A Appendix: Commands and Code Reference **46**
A.1 A.1 Domain Controller Setup . . . 46
A.2 A.2 Windows Client Configuration . . . 46
A.3 A.3 Linux Web Server (WEB01) Setup . . . 47
A.4 A.4 Kali Linux Setup . . . 47
A.5 A.5 Web Server Assessment (WEB01) . . . 47
A.6 A.6 Domain Controller Assessment (DC01) . . . 48
A.7 A.7 Windows Client Assessment (WIN-CLIENT) . . . 48
A.8 A.8 Full Lab Discovery Scan . . . 48
A.9 A.9 Log Correlation . . . 49

List of Figures

1 Lab network diagram . . . 8
2 Server Manager dashboard on the domain controller . . . 10
3 AD DS role health on the domain controller . . . 11
4 Local Server properties of the domain controller . . . 11
5 Sign-in screen of the domain controller . . . 12
6 Users in the labUsers organisational unit . . . 13
7 Command-line check of accounts and groups . . . 13
8 Static IP and DNS settings on the Windows client . . . 14
9 Domain join blocked on Windows 10 Home . . . 16
10 Ubuntu Server installation complete on WEB01 . . . 17
11 Netplan configuration on WEB01 . . . 18
12 Ping from WEB01 to the Windows client . . . 18
13 Ubuntu version on WEB01 . . . 18
14 Apache version on the web server . . . 19
15 Homepage opened from Kali . . . 20
16 Homepage opened from the Windows client . . . 20
17 Ping from Kali to the Windows client . . . 21
18 Ping from the Windows client to Kali . . . 22
19 Nmap service discovery results . . . 25
20 Evidence for finding F1 – Nikto scan . . . 27
21 Evidence for finding F1 – manual header check . . . 27
22 Supporting evidence for F1 – WhatWeb fingerprint . . . 28
23 Supporting evidence for F1 – Gobuster directory scan . . . 28
24 Evidence for finding F2 – anonymous SMB null session . . . 30
25 Domain information recovered through the null session . . . 30
26 Deeper enumeration refused in enum4linux-ng . . . 31
27 Deeper enumeration blocked despite the null session . . . 31
28 Nmap SMB script scan of the domain controller . . . 32
29 Anonymous LDAP bind refused . . . 32
30 Evidence for finding F3 – WIN-CLIENT port filtering . . . 33
31 Web access log summary of requests from Kali . . . 36
32 Web access log requests grouped by User-Agent . . . 36
33 Windows Security log: logon events from Kali . . . 37
34 Anonymous logons and their source address . . . 37
35 Time zone and clock on the domain controller . . . 38
36 Check result before the improvement . . . 40
37 Check result after the improvement . . . 41
38 After the improvement: target and domain information . . . 42

39 After the improvement: deeper enumeration still refused 43

List of Tables

1 Objectives and where each is addressed in this report 5
2 Steps followed in this project 6
3 Networks used in the lab, their purpose and whether they were tested 7
4 Lab machines, operating systems and IP addresses 7
5 Role of each lab machine and the reason it is needed 8
6 Accounts created in the domain, their group and the reason for each 14
7 Environment checks: what I did, why, and the outcome 23
8 Assessment scope and rules 24
9 Tools used, what each does and why it was chosen 24
10 Findings overview with affected system, severity and evidence 25
11 Log sources reviewed and why each was checked 34
12 My actions matched to the log entries they produced 35
13 Prioritised recommendations, the risk each reduces and the finding it relates to . 39
14 Before-and-after comparison of the repeated check 40

1 Project Overview and Objectives

1.1 Overview

This report documents the design, build, security assessment and hardening of a small virtual lab created for the NEST 2026 Cohort Final Project. The lab imitates a minimal small-office network. It contains a Windows Active Directory domain controller (referred to as "the DC" in this report), a Windows workstation and a Linux web server, together with a Kali Linux machine that I used as my assessment workstation. All four machines run as virtual machines on one host and are connected through an isolated host-only network, `192.168.56.0/24`, so nothing in the lab can reach a production network.

After building the environment, I checked that its parts could work together: the machines could reach each other, the workstation could find the domain controller through DNS, and the web server served its site. I planned to join the workstation to the domain, but this could not be completed because the workstation runs Windows 10 Home, which cannot join a domain (Section 7). I then carried out a focused security assessment from Kali against the three lab systems, recorded the findings with evidence, and reviewed the Windows and Linux logs to see which of my activities were visible to a defender. Finally, I recommended and prioritised improvements, implemented one of them, and repeated the original check to see whether it worked.

The report follows the order of that work. Section 2 describes the environment design, Section 3 the installation and configuration, Section 4 the assessment, Section 5 the log evidence, and Section 6 the recommendations with verification. Section 7 covers the challenges, limitations and lessons learned. Each section begins with a short note on what I did and the steps I followed.

1.2 Objectives

1. Build a small isolated lab with a Windows domain controller, a Windows client and a Linux web server.
2. Perform a focused security assessment and document at least three findings with evidence.
3. Investigate the logs generated by the assessment and connect them to my tests.
4. Recommend, prioritise and implement at least one improvement, then verify the result.

Table 1 shows where each objective is addressed, so a reader can go straight to the relevant evidence.

Table 1: Objectives and where each is addressed in this report

No.	Section	What is covered	Main evidence
1	2, 3	Design, installation and configuration of the four machines, and checks that they work together	Tables 4, 5 and 7; Figures 1–18
2	4	Scope, tools, method and three documented findings	Tables 9 and 10; Figures 19–30
3	5	Windows and Linux logs matched to the assessment activity	Tables 11 and 12; Figures 31–35
4	6	Prioritised recommendations, one implemented fix and a before-and-after check	Tables 13 and 14; Figures 36–39

1.3 Summary of Work and Steps Followed

I worked through the project in the order shown in Table 2. I built and checked each machine before moving on, so that any problem found later could be traced to the assessment and not to a broken lab.

Table 2: Steps followed in this project

Step	Stage	What I did	Section
1	Planning and design	Chose four virtual machines, two networks (an isolated host-only lab network and a NAT network for downloads) and a fixed address for each machine.	2
2	Domain controller	Installed Windows Server 2022, set a static IP, installed AD DS and DNS, created the forest `michael.corp`, and created an organisational unit, three users and a group.	3
3	Windows client	Installed Windows 10, set a static IP with the DC as DNS server, added a ping rule, ran the pre-join checks and tried to join the domain. The join was blocked by the Home edition.	3
4	Linux web server	Installed Ubuntu Server, configured both adapters with netplan, installed Apache and deployed a one-page homepage.	3
5	Kali and connectivity	Gave Kali a static lab address and tested ping in both directions with the client.	3
6	Environment checks	Ran the checks in Table 7 to confirm that the lab works before testing.	3
7	Assessment	Fixed the scope, scanned the lab with Nmap, enumerated the services found, confirmed each weakness by hand and recorded three findings.	4
8	Log review	Matched my test activity to Windows and Apache log entries by time and source address.	5
9	Hardening	Ranked recommendations, implemented one fix and repeated the original check.	6
10	Reflection	Recorded the problems I met, the limits of the lab and what I learned.	7

1.4 Rules of Engagement

- The lab uses an isolated host-only network with no route to production networks.
- Only lab-only users, groups and sample data are used.
- Testing was performed only against systems I own and built for this project.

1.5 Teamwork and Shared Resources

I completed this project individually. No lab resources were shared with a teammate, and all configuration, testing, findings and writing in this report are my own work.

2 Environment Design and Network Diagram

What I did in this section. Before building anything, I decided which machines the lab needs and how they connect. I (1) chose one machine for each role: domain controller, workstation,

web server and attacker; (2) chose an isolated host-only network for all lab traffic and a separate NAT network for downloads; (3) gave every machine a fixed address; and (4) drew the diagram in Figure 1.

2.1 Design Summary

The lab is built around one isolated host-only network, `192.168.56.0/24`. It carries all traffic between the machines and is the only network in scope for testing. The DC provides Active Directory and DNS for the domain `michael.corp` (NetBIOS name `MICHAEL`). The Windows client was intended to join that domain, but it runs Windows 10 Home, which cannot join a domain, so it stayed in a workgroup (Section 7). The Linux web server hosts the sample website. Kali Linux sits on the same segment as the assessment workstation, which keeps the testing realistic: it behaves like a machine that has gained a foothold on the internal network.

Kali, the Windows client and the web server also have a second adapter on a VMware NAT network (`192.168.128.0/24`). I used it only to download tools, packages and updates. It is outside the assessment scope and carries no traffic between the lab machines. The DC has no NAT adapter. Table 3 summarises the two networks.

Table 3: Networks used in the lab, their purpose and whether they were tested

Network	Subnet	Purpose and reason for use	Scope	Machines
Host-only lab	`192.168.56.0/24`	Isolated segment for all traffic between lab machines. It was chosen so that testing can never reach a production network.	In scope	All four
VMware NAT	`192.168.128.0/24`	Internet access for downloading tools, packages and updates. It is kept separate from the lab segment.	Out of scope	Kali, WIN-CLIENT, WEB01

2.2 Machines and Roles

The lab contains four machines. Table 4 lists their operating systems and addresses, and Table 5 explains the role each machine plays and why it is needed. Every machine has an address on the host-only lab network. Kali, the Windows client and the web server also have a second address on the NAT network, which is used for downloads.

Table 4: Lab machines, operating systems and IP addresses

Hostname	Operating system	Lab IP (host-only)	NAT IP (downloads only)
`DC01`	Windows Server 2022 Standard (Evaluation)	`192.168.56.30`	None
`WIN-CLIENT`	Windows 10 Home, version 22H2	`192.168.56.20`	`192.168.128.129`
`WEB01`	Ubuntu Server 26.04 LTS	`192.168.56.40`	DHCP, `192.168.128.0/24`
`kali`	Kali Linux	`192.168.56.10`	`192.168.128.128`

The Windows computer name of the domain controller is `WIN-R96H96QO5CC` (visible in Figures 38 and 37); this report refers to it as `DC01` for readability.

Table 5: Role of each lab machine and the reason it is needed

Machine	Role	Why it is needed
Domain controller (`DC01`)	Runs Active Directory and DNS for the domain `michael.corp`.	It holds the accounts that I assess and provides DNS for the lab.
Windows client (`WIN-CLIENT`)	Windows 10 Home workstation, intended as a domain member but left in a workgroup.	It represents an ordinary employee machine on the lab network. It could not join the domain because Windows 10 Home does not support domain join.
Linux web server (`WEB01`)	Runs the Apache HTTP Server and serves the sample website.	It is the web target of the assessment and produces the web server logs that I review later.
Kali Linux (`kali`)	Assessment workstation that holds the testing tools.	It acts as an internal attacker on the lab segment, so every test starts from one known address.

2.3 Network Diagram

Figure 1 shows how the machines connect. The diagram separates the isolated lab segment, used for all testing, from the NAT network, used only for downloads.

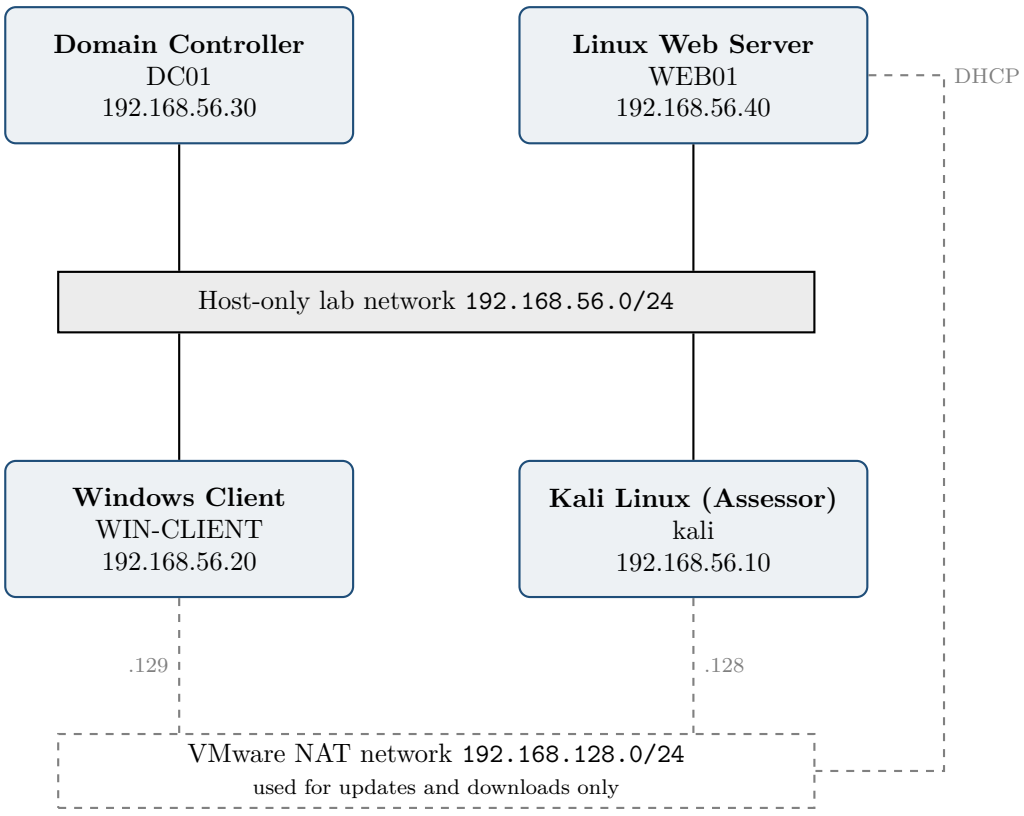


Figure 1: Lab network diagram. I drew this diagram to show which machines share the isolated lab segment and which have internet access. Solid lines are the isolated lab network used for all testing. Dashed lines are the NAT adapters on Kali, the Windows client and the web server, used only for downloads. The diagram shows that the DC has no internet path at all.

2.4 How the Machines Communicate

All lab traffic stays on the `192.168.56.0/24` segment. The DC runs DNS for `michael.corp`, and the Windows client uses its address (`192.168.56.30`) as its DNS server. This lets the client locate the domain controller, which I confirmed with `nslookup` and `nltest` (Section 3). The client could not complete the domain join itself because of its Windows edition. Domain sign-in on a joined machine would use standard Active Directory protocols (Kerberos, LDAP and SMB). The Linux web server listens on HTTP (port 80), and both the client and Kali reach it using its lab address. Kali has no special access: it reaches the other machines only through the same network a normal workstation would use.

No default gateway is configured on the lab adapters. The NAT adapters on Kali, the client and the web server carry the default route, so the lab network stays isolated while those three machines can still download updates when needed.

3 Installation and Configuration

What I did in this section. I built the lab in the following order and checked each machine before starting the next:

1. Created the virtual machines in VMware Workstation and attached them to the host-only and NAT networks.
2. Installed Windows Server 2022, set its static address, installed AD DS and DNS, promoted it to a domain controller and created the organisational unit, users and group.
3. Configured the Windows 10 client with a static address and the DC as DNS server, ran the pre-join checks and tried to join the domain.
4. Installed Ubuntu Server on `WEB01`, fixed its network with netplan, installed Apache and deployed a one-page homepage.
5. Gave Kali a static lab address and tested ping in both directions.
6. Ran the environment checks in Table 7.

3.1 Virtualisation Platform

All four machines run as virtual machines in VMware® Workstation 17 Pro on a single host. I chose a desktop hypervisor because it lets me create an isolated host-only network and a separate NAT network without any physical equipment. The host-only adapter of each machine is attached to a VMware host-only network, which isolates the lab segment from the physical network. The NAT adapters on Kali, the Windows client and the web server use VMware's NAT network, which provides internet access for downloads only.

3.2 Windows Domain Controller

The DC, `DC01`, runs Windows Server 2022 Standard (Evaluation) on the host-only lab network at `192.168.56.30`. The forest root domain is `michael.corp` and its NetBIOS domain name is `MICHAEL`. I built the server in four steps [3]:

1. Installed Windows Server 2022 and set the static IP address `192.168.56.30`. A domain controller needs a fixed address because the clients use it as their DNS server.

2. Installed the Active Directory Domain Services (AD DS) role through Server Manager. This role provides the directory that stores users, groups and computers.
3. Promoted the server to a domain controller by creating a new forest with the root domain name `michael.corp`. The DNS Server role was installed during this step because Active Directory depends on DNS to locate its services.
4. Created an organisational unit, users and a security group (Section 3.2.1) so that the domain contained realistic accounts to sign in with and to assess.

After promotion I opened Server Manager to confirm that the roles were present and healthy (Figures 2 and 3) and checked the Local Server page for the domain and network settings (Figure 4). Finally, I looked at the sign-in screen to confirm that the server now authenticates against its own domain (Figure 5).

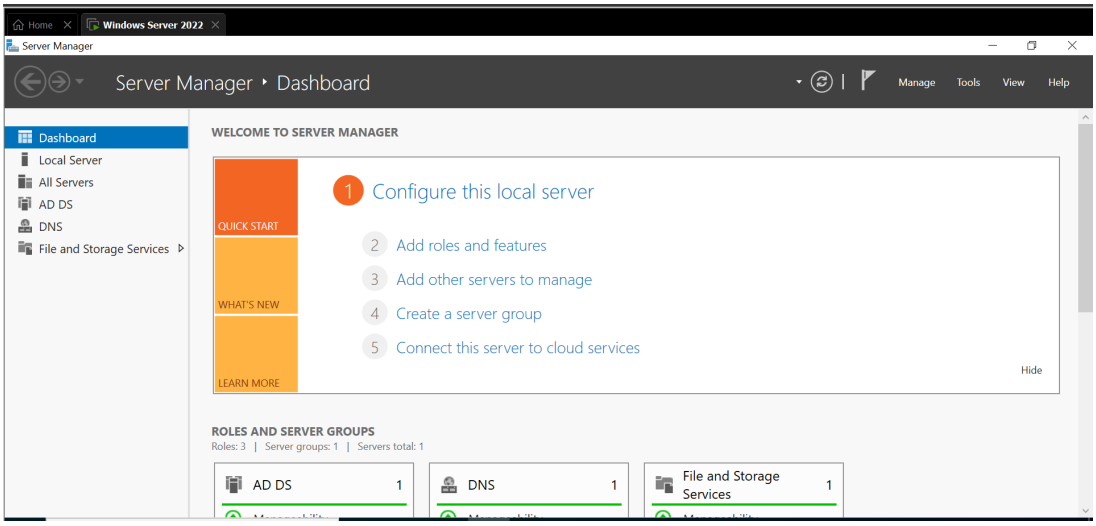


Figure 2: Server Manager dashboard on the domain controller. I opened Server Manager on the DC after promotion to check which roles were installed. The navigation pane and the role tiles list AD DS, DNS and File and Storage Services, with one server in the server group. This confirms that the server holds the Active Directory and DNS roles the lab depends on.

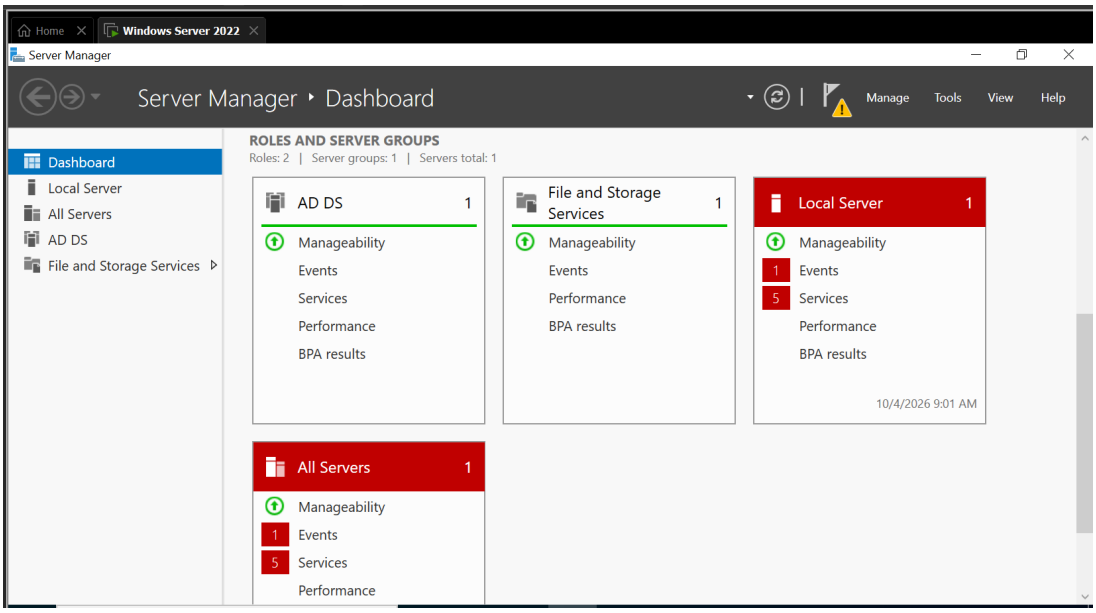


Figure 3: AD DS role health on the domain controller. I scrolled down to the role tiles in Server Manager to check the health of the AD DS role. The AD DS tile shows a green manageability status and no events or services in red, so the role installed and is running normally.

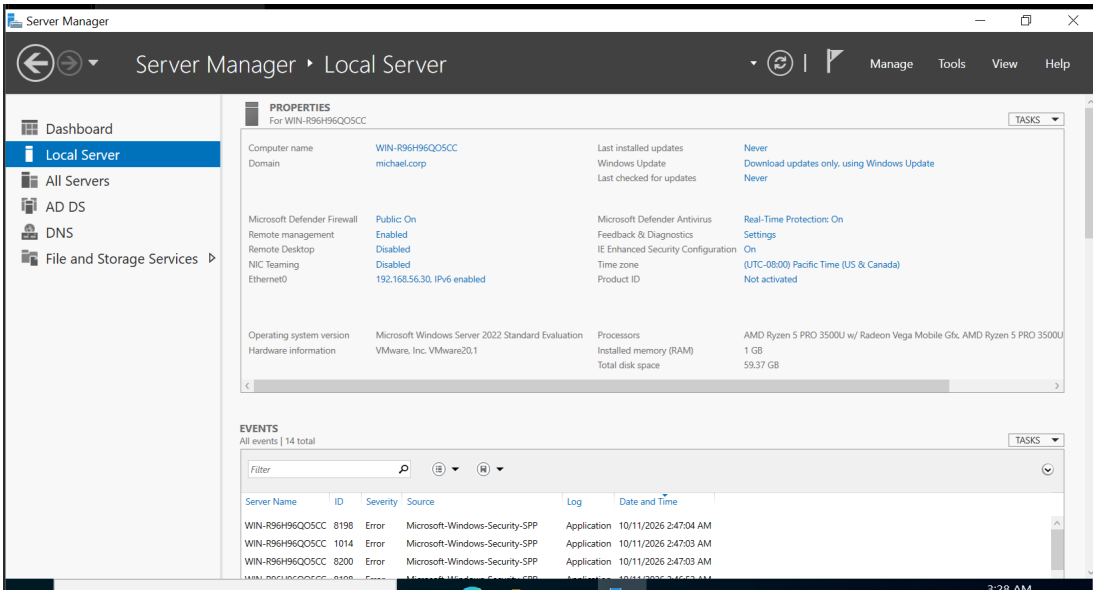


Figure 4: Local Server properties of the domain controller. I opened Server Manager > Local Server to record the server's configuration. The properties show that the server is a member of the `michael.corp` domain, that its network adapter has the address `192.168.56.30`, and that it runs Windows Server 2022 Standard Evaluation as a VMware virtual machine with 1 GB of memory and about 59 GB of disk space. Microsoft Defender Firewall and real-time antivirus protection are on, and the product is not activated because it is an evaluation copy.

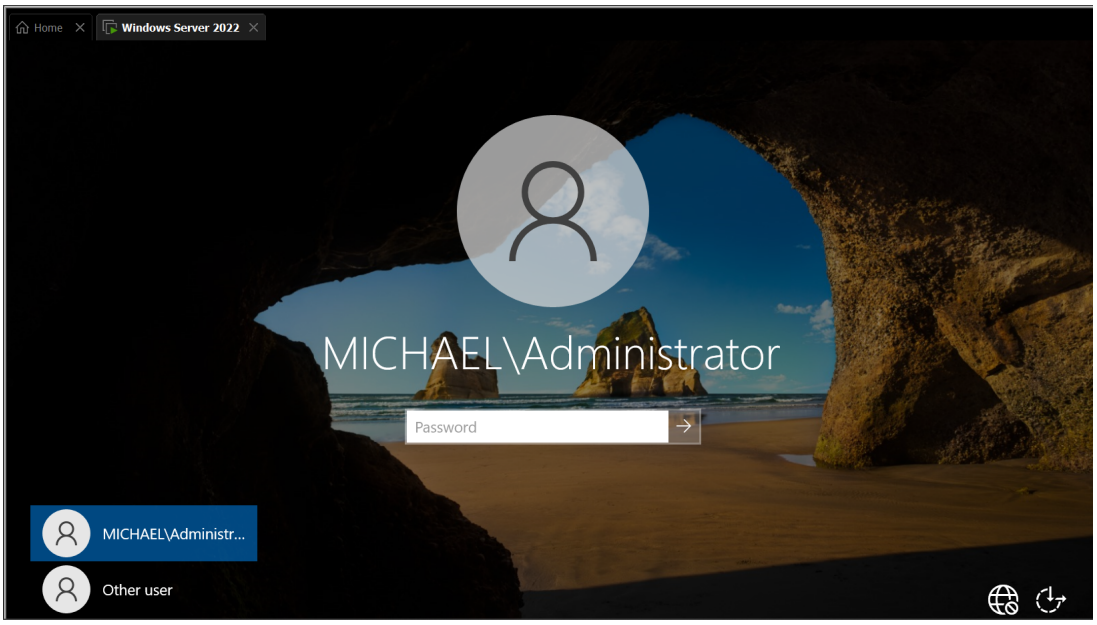


Figure 5: Sign-in screen of the domain controller. I checked the sign-in screen after the promotion reboot to confirm that the server now uses domain accounts. The screen shows "Sign in to: MICHAEL" and offers `MICHAEL\Administrator` and an "Other user" option, so the sign-in is against the `michael.corp` domain (NetBIOS name `MICHAEL`) and no longer against a local account database.

3.2.1 Organisational Unit, Users and Groups

Using Active Directory Users and Computers, I created an organisational unit named `labUsers` in the `michael.corp` domain and added three user accounts to it: `Aerixis`, `hadmin` and `svc_web` (Figure 6). I also created a security group named `Staff` and made all three accounts members of it. I used a group so that permissions can later be given to the group rather than to each person individually.

To confirm the result independently of the graphical tool, I ran `net user` and `net groups` in a Command Prompt on the DC (Figure 7). Table 6 records each account, its group and the reason it was created.

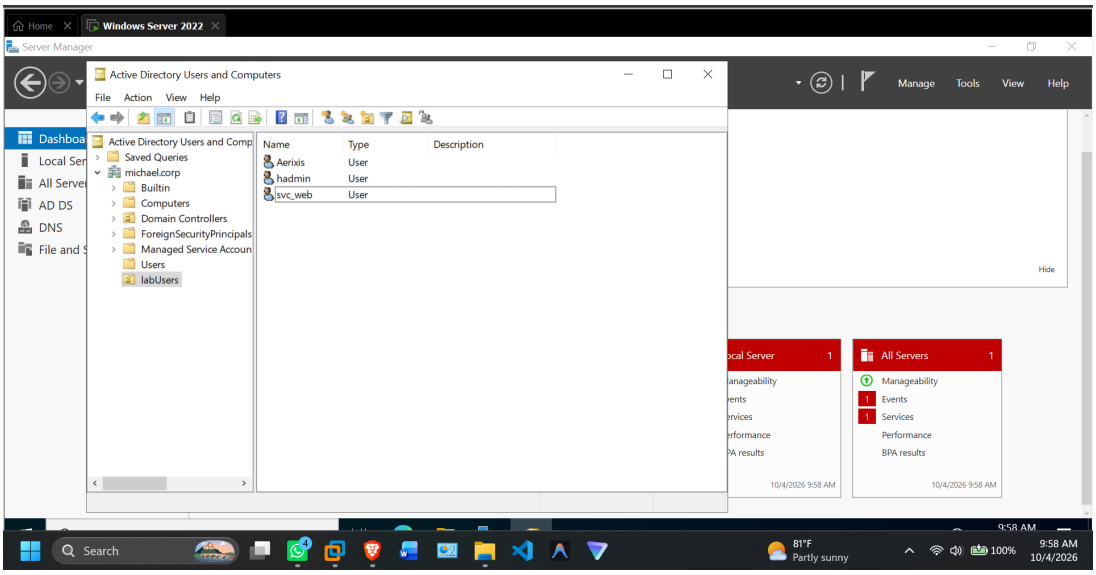


Figure 6: Users in the labUsers organisational unit. I opened Active Directory Users and Computers and selected the `labUsers` organisational unit to check the accounts I had created. The right-hand pane lists three user objects: `Aerixis`, `hadmin` and `svc_web`. The accounts exist in the `michael.corp` domain and are ready to be used for the assessment.

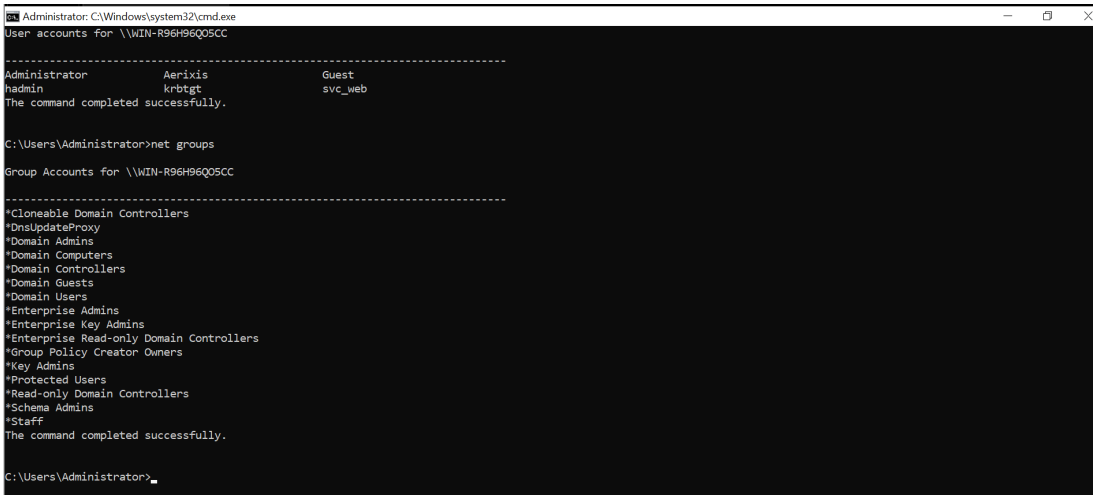


Figure 7: Command-line check of accounts and groups. I ran `net user` and `net groups` in a Command Prompt on the DC to verify the directory contents without using the graphical tool. `net user` lists `Aerixis`, `hadmin` and `svc_web` next to the built-in `Administrator`, `Guest` and `krbtgt` accounts. `net groups` lists the default domain groups together with the new `Staff` group.

Listing 1: Equivalent PowerShell commands for creating a user and adding it to Staff (I used the graphical tool)

```
New-ADUser -Name "Helpdesk Admin" -SamAccountName "hadmin" ‘
  -Path "OU=labUsers,DC=michael,DC=corp" ‘
  -AccountPassword (Read-Host -AsSecureString) -Enabled $true
Add-ADGroupMember -Identity "Staff" -Members "hadmin"
```

Table 6: Accounts created in the domain, their group and the reason for each

Account	Group	OU	Purpose	Why it was created
`Aerixis`	Staff	labUsers	Standard domain user	Intended for a domain sign-in test on the client, which was not possible on Windows 10 Home
`hadmin`	Staff	labUsers	Delegated helpdesk account	Represents a helpdesk user, a typical target for privilege checks
`svc_web`	Staff	labUsers	Service account	Represents the identity a web application uses to talk to the domain

3.3 Windows Client and Domain Join Attempt

The Windows 10 client has two adapters. On the host-only adapter I set a static address of `192.168.56.20` with a `/24` prefix, and I set its preferred DNS server to `192.168.56.30`, the address of the DC (Figure 8). The static address keeps the client at a known place for the assessment, and because it is set manually it does not change after a restart. The DNS setting matters most for a domain join: the client finds the domain by asking the DC's DNS server, so without it the join fails.

I also added a firewall rule that allows ping requests from the lab subnet only. The rule lets me test connectivity without opening ping to other networks. Listing 2 shows the `netsh` commands for the address, the DNS server and the rule.

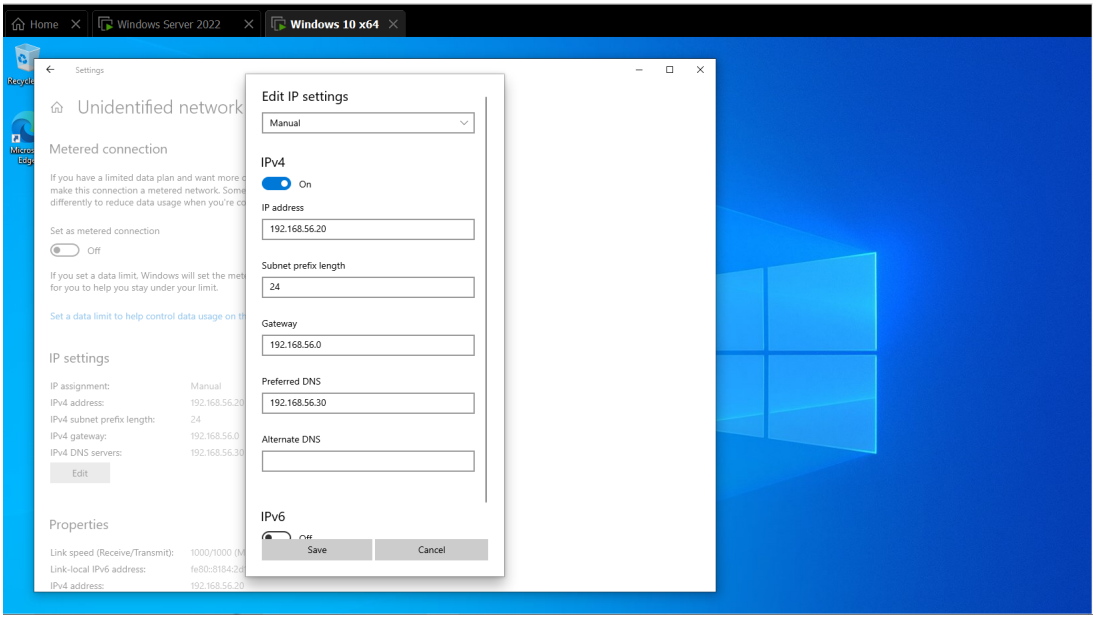


Figure 8: Static IP and DNS settings on the Windows client. I configured the client's host-only adapter manually so that it has a fixed address and uses the DC for name resolution. The settings show the assignment as Manual, the IPv4 address `192.168.56.20`, the subnet prefix length `24` and the DNS server `192.168.56.30`. The client is now on the lab network and can look up the `michael.corp` domain.

Listing 2: netsh commands for the client address, DNS server and ping firewall rule (run as administrator)

```
netsh interface ip set address name="Ethernet1" static 192.168.56.20 255.255.255.0
netsh interface ip set dns name="Ethernet1" static 192.168.56.30 validate=no
netsh advfirewall firewall add rule name="Allow ICMPv4 In (Lab)" protocol=icmpv4:8,any dir=in
    action=allow remoteip=192.168.56.0/24
```

3.3.1 Checks Before Joining the Domain

Before joining, I checked that the client could reach and query the DC (Listing 3). `ipconfig /all` showed the address `192.168.56.20` and the DNS server `192.168.56.30` on the host-only adapter. A ping to the DC returned 4 of 4 replies with TTL 128, which matches a Windows host. `nslookup michael.corp` returned `192.168.56.30`, so the DC's DNS server answers for the domain. The first `nslookup` attempt printed a timeout line and "Server: UnKnown". This happens because the DNS server's own address has no reverse (PTR) record, so it did not affect the result. `nltest /dsgetdc:michael.corp` located the DC at `192.168.56.30` and listed the writable, Kerberos (KDC) and Global Catalog roles in its flags, which shows that the client can find a domain controller for `michael.corp`.

When I first set the DNS server with `netsh`, it printed "The configured DNS server is incorrect or does not exist." This is a validation warning from `netsh`. The setting was applied, as `ipconfig /all` confirmed, and I repeated the command with `validate=no` (Listing 2).

Listing 3: Checks run on the client before the domain join

```
ipconfig /all
ping 192.168.56.30
nslookup michael.corp
nltest /dsgetdc:michael.corp
```

3.3.2 Domain Join Attempt and Result

I followed a video tutorial [4] to learn the join procedure: open System Properties, choose **Change...** on the Computer Name tab, select **Domain**, enter the domain name and supply domain administrator credentials. In the Computer Name/Domain Changes window on my client, however, the **Domain** option and its text box were greyed out and only the **Workgroup** option (set to `WORKGROUP`) could be selected. The window states that a computer running this edition of Windows 10 cannot join a domain (Figure 9). Settings > About showed that the client runs **Windows 10 Home**, version 22H2 (OS build 19045.2006).

Windows 10 Home does not support joining an Active Directory domain. Joining requires Windows 10 Pro, Enterprise or Education. Because my client only has the Home edition, **I could not join it to the domain**. The pre-join checks above show that the network, DNS and DC discovery were all working, so the edition was the only blocker. Listing 4 shows the command I would use on a supported edition. I did not run it successfully, and the client stayed in the workgroup `WORKGROUP`. Section 7 describes how this affects the project.

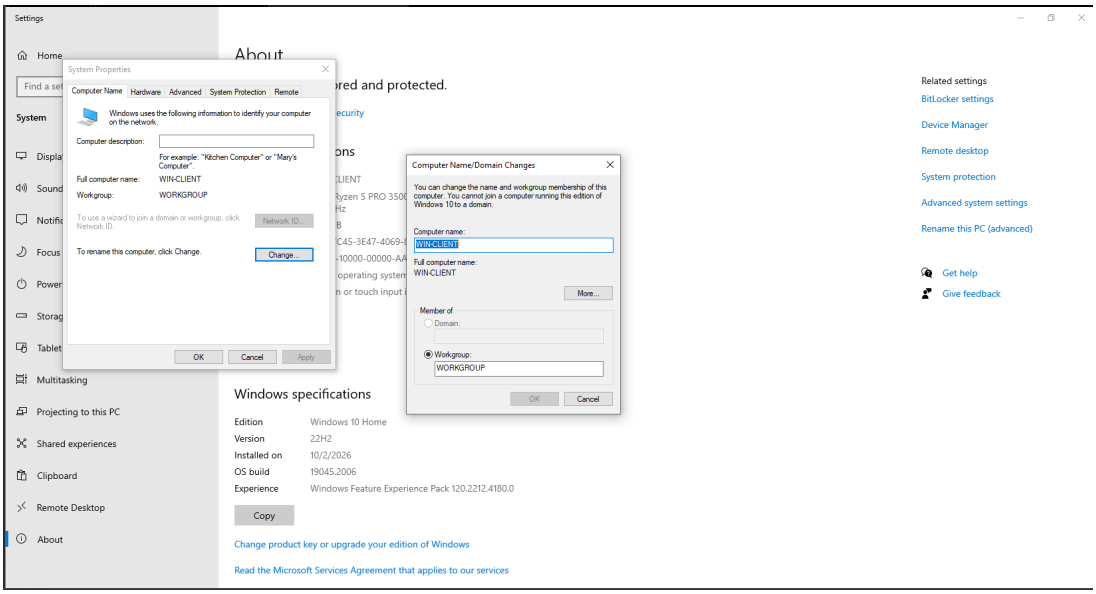


Figure 9: Domain join blocked on Windows 10 Home. I opened the Computer Name/Domain Changes window on `WIN-CLIENT` to join it to `michael.corp`. The **Domain** option and its text box are greyed out and the **Workgroup** option is selected with the value `WORKGROUP`. The message at the top of the window says that a computer running this edition of Windows 10 cannot join a domain. In the background, Settings > About lists the edition as Windows 10 Home, version 22H2. This shows that the join was blocked by the Windows edition and not by the network or the DC.

Listing 4: Command for joining the domain on a supported edition (not completed on Windows 10 Home)

```
Add-Computer -DomainName michael.corp -Credential MICHAEL\Administrator -Restart
```

3.4 Linux Web Server

`WEB01` has two adapters. The first (`ens33`) is attached to the VMware NAT network and receives its address by DHCP, which I use to download packages and updates. The second (`ens37`) is attached to the host-only network at the static address `192.168.56.40`, where both the Windows client and Kali can reach the website over HTTP on port 80.

I built the server as a new virtual machine in VMware Workstation and installed Ubuntu Server from the installation ISO using the text-based installer. I chose Ubuntu Server because it has no desktop environment, which keeps the machine small and reduces the number of services that could be attacked, and because it is a common choice for hosting websites. When the installer finished, it displayed the message “Installation complete!” and offered the `Reboot Now` option (Figure 10).

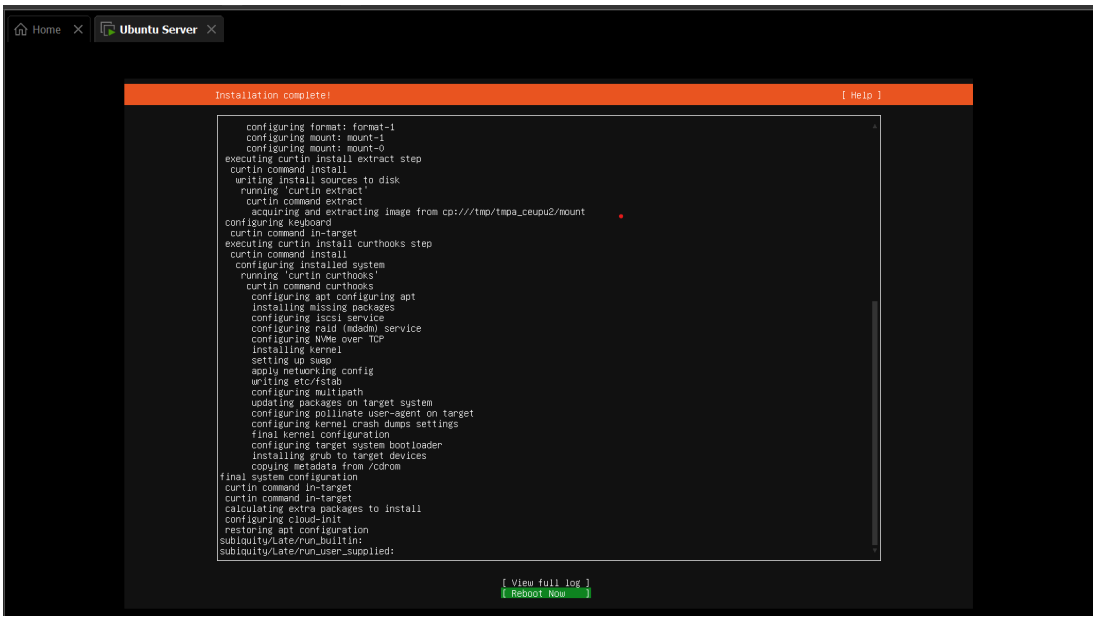


Figure 10: Ubuntu Server installation complete on WEB01. I took this screenshot at the end of the Ubuntu Server installation on `WEB01` to show that the installer finished. The orange banner reads "Installation complete!" and the log lists the final steps, including installing the kernel, writing `/etc/fstab`, installing GRUB to the target device and the final system configuration. The log ends without any error message, and the `Reboot Now` button is highlighted, so the next step was to restart into the newly installed system.

3.4.1 Network Configuration

After the first boot I ran `ip a` and `ping google.com`. Neither adapter had an IPv4 address, only IPv6 link-local addresses, and the ping failed with "Temporary failure in name resolution". Ubuntu Server configures its network with netplan, and the installer had not set up the second adapter. I therefore edited `/etc/netplan/50-cloud-init.yaml` so that `ens33` (NAT) uses DHCP and `ens37` (host-only) has the static address `192.168.56.40/24` (Figure 11 and Listing 5). I gave `ens37` no gateway, so the NAT adapter carries the default route and the lab segment stays isolated, as on the other machines. I then applied the file with `sudo netplan apply`.

Listing 5: Netplan configuration for WEB01

```
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: true
    ens37:
      dhcp4: false
      addresses:
        - 192.168.56.40/24
```

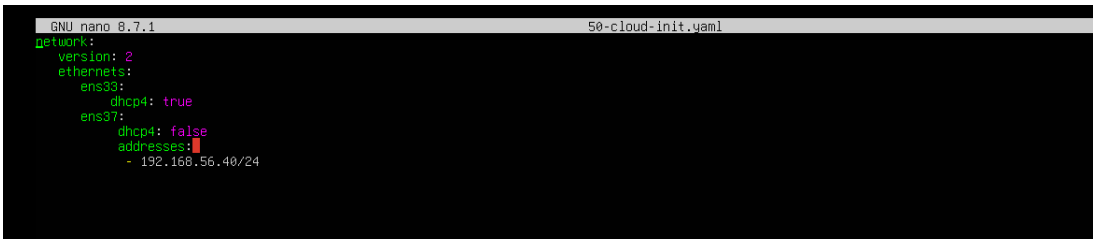


Figure 11: Netplan configuration on WEB01. I opened `/etc/netplan/50-cloud-init.yaml` in nano on `WEB01` to configure both adapters. The file sets `dhcp4: true` on `ens33`, the NAT adapter, and `dhcp4: false` with the address `192.168.56.40/24` on `ens37`, the host-only adapter. There is no gateway entry on `ens37`.

To check the result, I pinged the Windows client (`192.168.56.20`) from `WEB01` (Figure 12). All 4 packets were answered with 0% packet loss and a TTL of 128, which is the default for Windows, so the replies came from the client. The round-trip time averaged about 22 ms, with one slower reply of 86 ms and the others close to 1 ms. This shows that the static address works and that `WEB01` can reach the other lab machines on the host-only network.

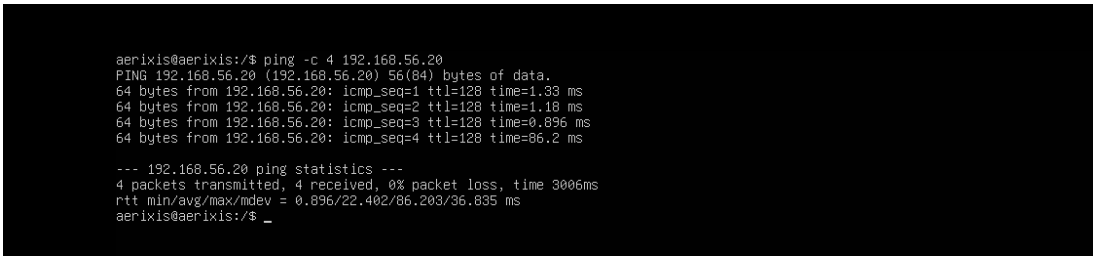


Figure 12: Ping from WEB01 to the Windows client. I ran `ping -c 4 192.168.56.20` on `WEB01` to confirm that the host-only adapter works. The result was 4 packets transmitted, 4 received and 0% packet loss, with round-trip times from 0.896 ms to 86.2 ms (average 22.4 ms). The TTL of 128 shows that the Windows client answered.

3.4.2 Web Server Software

`WEB01` runs Ubuntu Server 26.04 LTS (codename "resolute"), confirmed with `lsb_release -a` (Figure 13). I installed the Apache HTTP Server from the Ubuntu repositories and enabled it at boot (Listing 6). `apache2 -v` reports version 2.4.66 (Figure 14). I chose Apache because it is widely used, is packaged by Ubuntu and writes a plain-text access log that is easy to match against my scans.



Figure 13: Ubuntu version on WEB01. I ran `lsb_release -a` on the web server to record the exact operating system. The output shows Ubuntu 26.04 LTS, release 26.04, codename resolute.

Listing 6: Installing and starting Apache

```
sudo apt update && sudo apt install -y apache2
sudo systemctl enable --now apache2
```

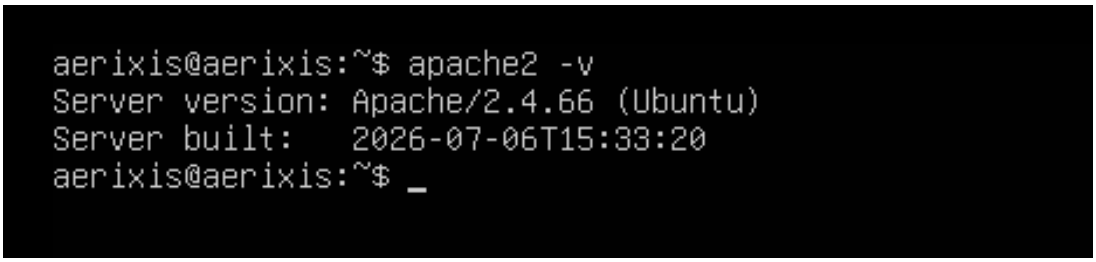


Figure 14: Apache version on the web server. I ran `apache2 -v` on the web server to record the exact Apache version, because the version determines which known vulnerabilities apply. The output shows Apache/2.4.66 (Ubuntu), built on 2026-07-06.

3.4.3 Homepage Deployment and Access Test

To give the assessment a site to test, I wrote a single-page homepage, `index.html`, with its CSS and JavaScript inside the same file, and placed it in Apache's default web root, `/var/www/html`, in place of the default page (Listing 7). I used an AI assistant (Claude, by Anthropic) to generate the HTML, CSS and JavaScript for this homepage, since its content was not the subject of the assessment – the page exists only to give the assessment tools something realistic to scan and enumerate. Apache serves static files directly from disk, so no restart was needed. Running `curl -I http://192.168.56.40` on `WEB01` returned `HTTP/1.1 200 OK` from Apache/2.4.66, which showed that the page was being served.

I then opened `http://192.168.56.40` in a browser from two other machines on the lab network: first from Kali, the assessment machine (Figure 15), and then from the Windows client (Figure 16). Testing from both machines shows that the web server is reachable from the attacker's address and from an ordinary workstation, so both can be used as starting points in the assessment.

Listing 7: Deploying the homepage and checking that Apache serves it

```
mv /home/aerixis/index.html /var/www/html/index.html
chown root:root /var/www/html/index.html
chmod 644 /var/www/html/index.html
curl -I http://192.168.56.40
```

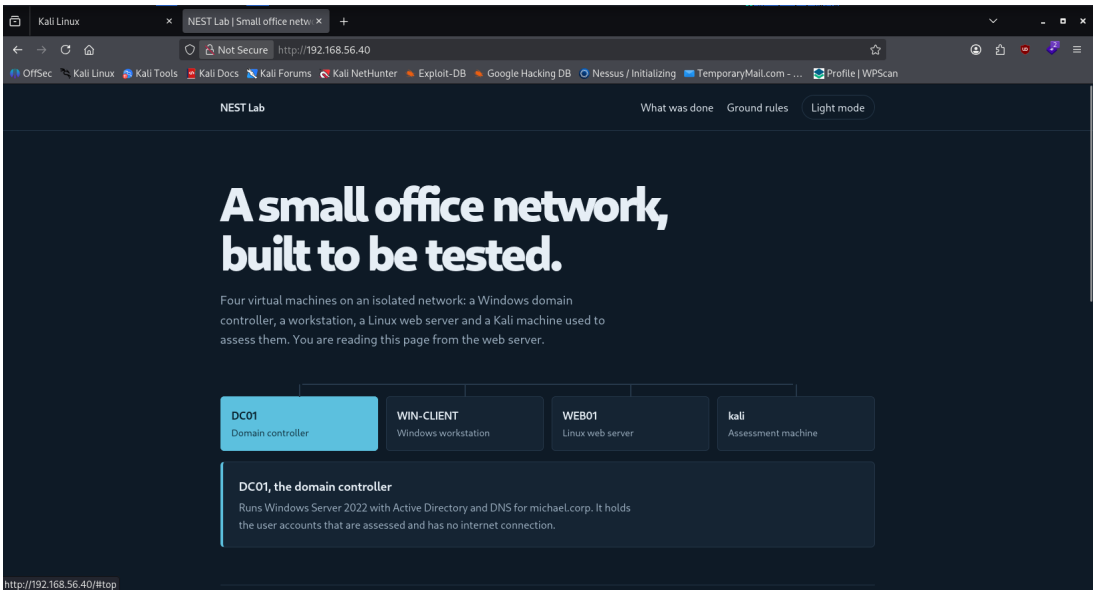


Figure 15: Homepage opened from Kali. I opened `http://192.168.56.40` in a browser on Kali (`192.168.56.10`) to check that the web server answers requests from the assessment machine. The homepage served by Apache on `WEB01` loads, so port 80 is reachable from Kali across the lab network.

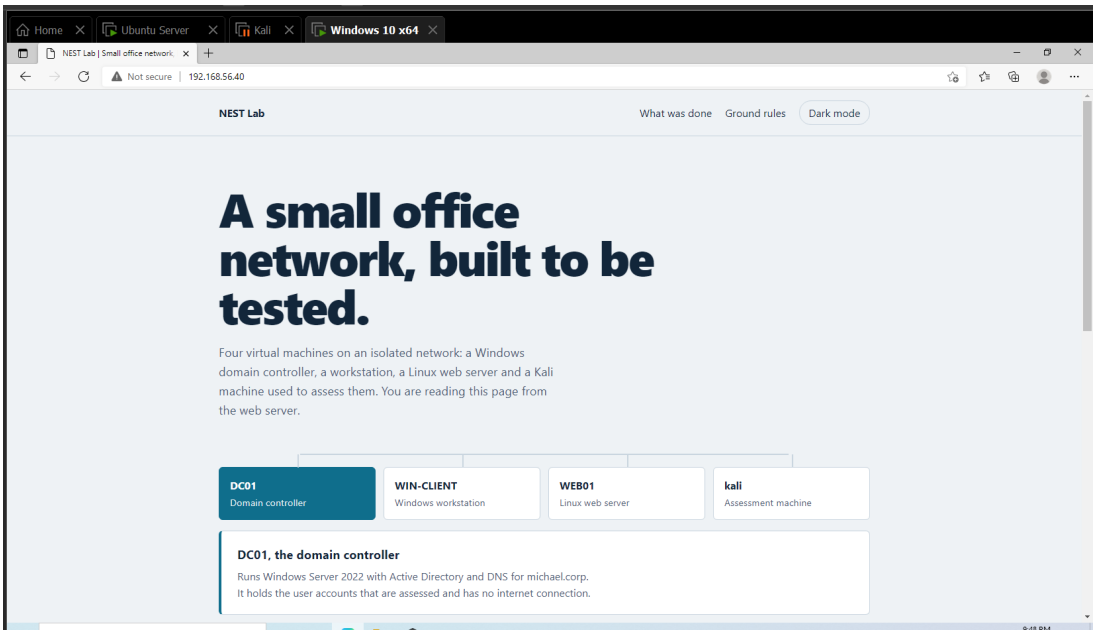


Figure 16: Homepage opened from the Windows client. I opened `http://192.168.56.40` in a browser on the Windows client (`192.168.56.20`) to check that the web server also answers requests from an ordinary workstation. The same homepage loads, so port 80 is reachable from the client across the lab network.

3.5 Static IP Configuration on Kali

Kali's host-only adapter (`eth1`) is given a fixed address, `192.168.56.10`, with NetworkManager (Listing 8). A fixed address makes every request in the logs traceable to Kali, which is needed in Section 5 to link log entries to my tests.

Listing 8: Persistent static IP on eth1 with NetworkManager

```
nmcli con add type ethernet ifname eth1 con-name eth1-static \
  ipv4.method manual ipv4.addresses 192.168.56.10/24
nmcli con up eth1-static
```

3.6 Connectivity Test Between Kali and the Windows Client

After configuring the second adapter on each machine (Listings 8 and 2), I tested two-way connectivity on the `192.168.56.0/24` lab network. I tested both directions because a firewall can block traffic in one direction only. Kali (`192.168.56.10`) and the Windows machine (`192.168.56.20`) each replied to ping requests from the other, with 0% packet loss (Figures 17 and 18).

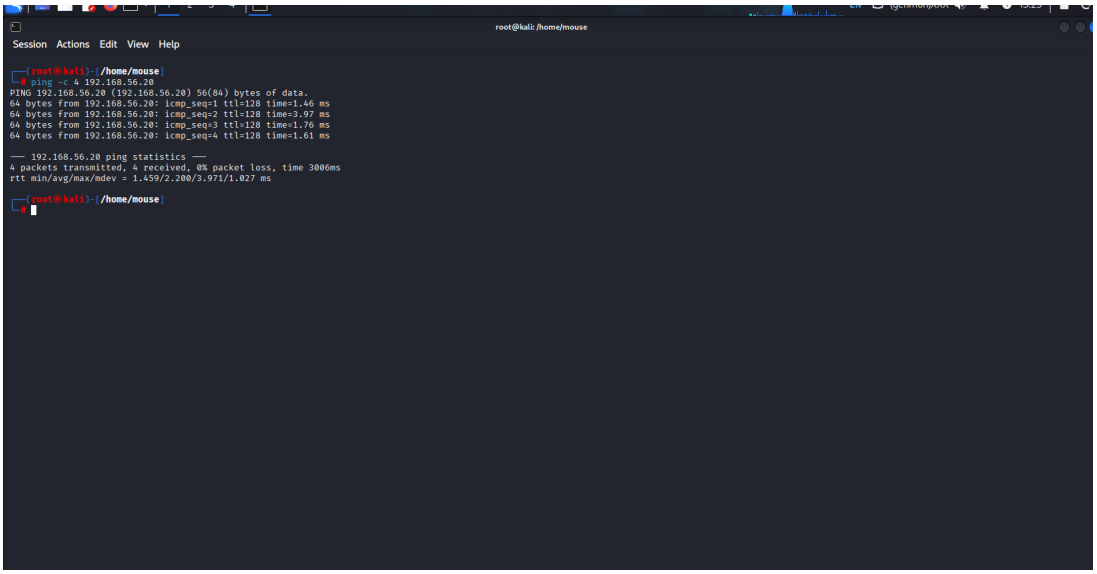


Figure 17: Ping from Kali to the Windows client. I ran `ping -c 4 192.168.56.20` from Kali (`192.168.56.10`) to check that Kali can reach the client on the lab network. The result was 4 packets transmitted, 4 received, 0% packet loss, with round-trip times between 1.5 and 4.0 ms (average 2.2 ms). The TTL of 128 is the default for Windows, so the replies came from the Windows client, and the client accepts ping from the lab subnet.

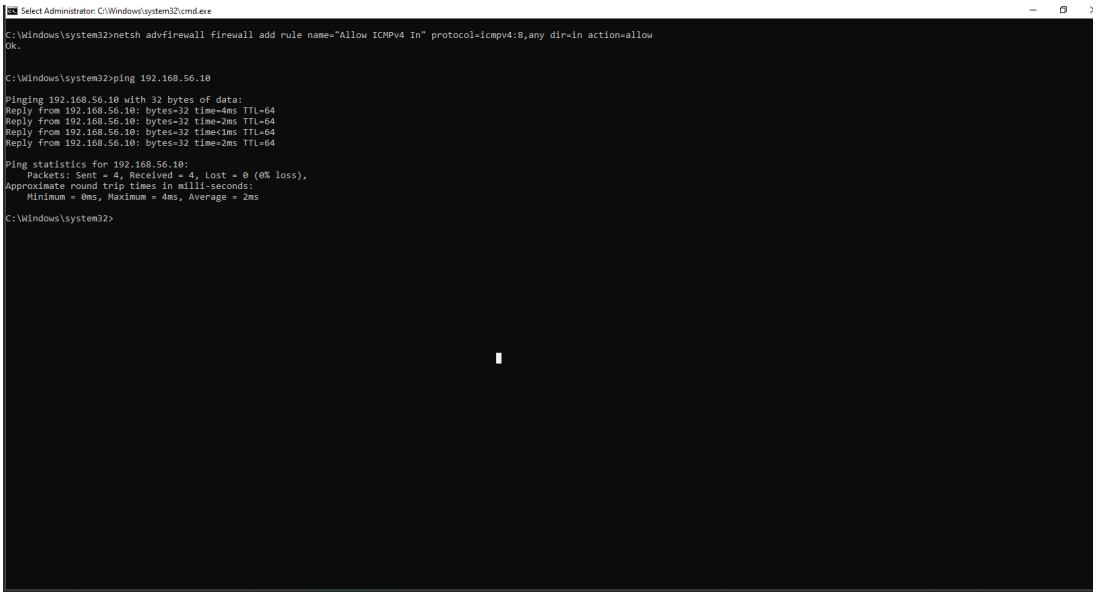


Figure 18: Ping from the Windows client to Kali. I pinged Kali (`192.168.56.10`) from the Windows machine to check the opposite direction. The result was 4 packets sent, 4 received, 0% loss, with an average round trip of 2 ms, so connectivity works both ways.

Both directions succeeded, which confirms that the two machines share the same virtual network segment and that the Windows firewall rule allowing ping from the lab subnet works. The TTL values in the replies (64 from Kali, 128 from Windows) match the default values of Linux and Windows hosts, which is consistent with each reply coming from the machine I expected.

3.7 Verification That the Environment Works

Before starting the assessment, I ran the checks in Table 7 to prove that the lab behaves as designed. Testing on a broken lab would give unreliable findings.

Table 7: Environment checks: what I did, why, and the outcome

Check	What I did	Why	Outcome
Client and Kali connectivity	Pinged `192.168.56.10` from the client and `192.168.56.20` from Kali (Figures 17, 18)	To confirm both machines share the lab segment and the firewall allows ping	Pass
Client reaches the DC	Pinged `192.168.56.30` from the client (4 of 4 replies, TTL 128)	To confirm the client has a network path to the DC	Pass
Web server reaches the client	Pinged `192.168.56.20` from `WEB01` (4 of 4 replies, TTL 128; Figure 12)	To confirm the web server's static address works on the lab segment	Pass
Client resolves the domain	Ran `nslookup michael.corp` on the client, which returned `192.168.56.30`	To confirm DNS on the DC answers the client, which a domain join relies on	Pass
Client locates the DC	Ran `nltest /dsgetdc:michael.corp` on the client	To confirm the client can find a writable domain controller	Pass
Web server reachable from Kali and the client	Opened `http://192.168.56.40` in a browser on Kali and on the Windows client (Figures 15, 16)	To confirm Apache is running and reachable from both the assessment machine and a workstation	Pass
Domain join and sign-in	Tried to join the client to `michael.corp` (Figure 9)	To confirm the client can join the domain and authenticate against the DC	Not completed: Windows 10 Home cannot join a domain

4 Assessment Scope, Methods and Findings

What I did in this section. With the lab working, I assessed it from Kali by following these steps:

1. Fixed the scope and rules (Table 8) and chose the tools (Table 9).
2. Ran an Nmap scan of the three lab systems to see which ports and services they expose.
3. Used the other tools to enumerate the web server and the domain services that the scan found.
4. Confirmed each possible weakness by hand, so that only verified issues are reported.
5. Recorded three findings with their evidence, severity and recommended fix (Table 10).

4.1 Scope

Table 8 defines what was and was not tested, so that the findings can be read in the right context.

Table 8: Assessment scope and rules

Item	Detail
In scope	The DC `DC01` (`192.168.56.30`), WIN-CLIENT (`192.168.56.20`) and WEB01 (`192.168.56.40`) on `192.168.56.0/24`
Assessment source	Kali Linux at `192.168.56.10`
Out of scope	Any host outside the three systems above, including the host machine and the NAT and internet side (`192.168.128.0/24`)
Authorisation	All systems were built by me and are owned by me for this project
Assessment window	11 October 2026, approximately 10:51–11:38 UTC (06:51–07:38 local Kali system time, UTC-4)

4.2 Tools Selected

Table 9 lists the tools I used, what each does and why I chose it.

Table 9: Tools used, what each does and why it was chosen

Tool	What it does	Why I chose it
Nmap [2]	Discovers hosts and lists open ports and service versions	To build an accurate picture of what each lab machine exposes before testing anything else
WhatWeb	Fingerprints the web technology stack (server software, OS, scripting)	To quickly identify what the web server is running before deeper testing
Nikto	Scans a web server for missing security headers, outdated software and common misconfigurations	To check for known web-server weaknesses beyond basic fingerprinting
Gobuster	Brute-forces directories and files on a web server	To check for hidden content, backup files or exposed sensitive paths
enum4linux-ng	Enumerates SMB and Active Directory information such as null-session access, domain details, users, groups and shares	To test whether the domain controller exposes directory information without credentials
rpcclient	Opens an interactive MS-RPC session over SMB for manual enumeration calls	To independently verify, call by call, whether SAMR and LSA enumeration are blocked on a null session
ldapsearch	Performs LDAP queries, including anonymous binds	To test whether the domain controller permits anonymous LDAP access as an alternate enumeration path

4.3 Methodology

I followed the phases of NIST SP 800-115 [1]: planning, discovery, attack (validation) and reporting. In planning I fixed the scope and rules in Table 8. In discovery I ran Nmap to find open ports and service versions, and then used the tools in Table 9 to enumerate the web server and the domain services that Nmap exposed. In validation I tested each possible weakness by hand before reporting it, so that every finding in this report is confirmed and not only a scanner result. In reporting I recorded each finding with its evidence, severity and fix.

Listing 9 shows the first scan I ran. It names only the three in-scope hosts, checks every port on each of them and records service versions, so later steps target only what is actually exposed.

Listing 9: Service discovery scan of the three in-scope hosts

```
nmap -sV -sC -p- -oN scans/lab_full.txt 192.168.56.20 192.168.56.30 192.168.56.40
```



Figure 19: Nmap service discovery results. I ran the scan in Listing 9 from Kali against the three lab systems to find every exposed service. The output lists the open ports and service versions on each host. The services that matter for the findings below are HTTP on WEB01 (Apache), SMB and LDAP on DC01, and the filtered ports on WIN-CLIENT.

4.4 Summary of Findings

Table 10 gives an overview of the findings. Each is described in full in the next subsection, with the evidence and the fix.

Table 10: Findings overview with affected system, severity and evidence

ID	System	Finding	Severity	Figures
F1	WEB01 (192.168.56.40)	Missing HTTP security headers (CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, HSTS)	Low–Medium	20–23
F2	DC01 (192.168.56.30)	Anonymous SMB null session permitted, exposing domain name and SID without credentials	Medium	24–29
F3	WIN-CLIENT (192.168.56.20)	Host firewall correctly blocks all common lateral-movement ports from the network (informational)	Informational	30

4.5 Detailed Findings

4.6 F1: Missing HTTP security headers on the web server

Affected system	WEB01, `192.168.56.40` (Apache/2.4.66 on Ubuntu)
Issue	The server sends no `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy` or `Strict-Transport-Security` header on any response.
Severity	Low–Medium
Evidence	See Figures 20–23. Confirmed by hand with `curl -I`, which returns only `Server`, `Date`, `Last-Modified`, `ETag`, `Accept-Ranges`, `Content-Length`, `Vary` and `Content-Type`; independently confirmed by Nikto, which flags the same five missing headers (plugin IDs 013587, 007342, 007352). A Gobuster directory brute-force against common backup and credential-file names (`.htaccess`, `.htpasswd` and their `.bak`/`.zip`/`.txt`/`.php` variants) returned 403 Forbidden on every entry, confirming no hidden content or directory listing is exposed alongside this finding.
Why it matters	None of these headers is present, so the browser has no instruction to block framing, enforce HTTPS or avoid MIME-sniffing a response. On its own this is low-exploitability, but it removes a layer of defence that would otherwise limit the impact of a future cross-site scripting or content-type confusion bug on the site.
Recommended fix	Add the headers in Apache, e.g. in a new `/etc/apache2/conf-available/security-headers.conf` enabled with `a2enconf` (the `headers` module must be enabled): `Header always set X-Content-Type-Options "nosniff"`, `Header always set X-Frame-Options "DENY"` and `Header always set Referrer-Policy "no-referrer"`. A Content-Security-Policy must be tested before use, because the homepage keeps its CSS and JavaScript inline and a strict `default-src 'self'` policy would block them; either move them into separate files or allow inline content explicitly. `Strict-Transport-Security` only takes effect over HTTPS, so it should be added once the site is served with TLS. Also apply the latest Apache security patches, since Nikto reports this installation (2.4.66) as slightly behind the newest release.



Figure 20: Evidence for finding F1 – Nikto scan. I ran Nikto against WEB01 to check for missing security headers and common misconfigurations. It flags the same five headers as missing: Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy and Strict-Transport-Security.



Figure 21: Evidence for finding F1 – manual header check. I ran `curl -I` against WEB01 to confirm the result of the scanner by hand. The response contains only the default headers and none of the five security headers, which rules out a Nikto false positive.



Figure 22: Supporting evidence for F1 – WhatWeb fingerprint. I ran WhatWeb against WEB01 to identify its technology stack before deeper testing. The output records the server software and operating system that the headers above belong to.

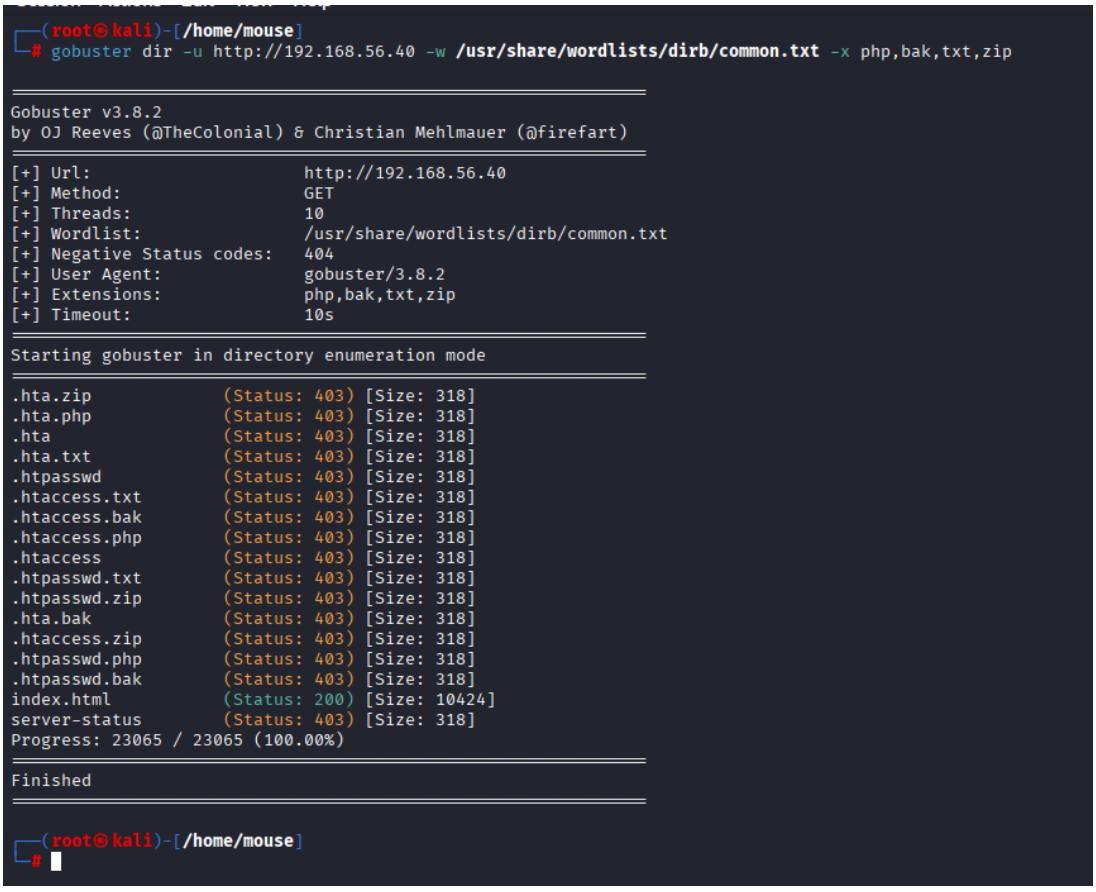


Figure 23: Supporting evidence for F1 – Gobuster directory scan. I ran Gobuster against WEB01 to check for hidden or sensitive files. Every backup and credential-file name tried returned 403 Forbidden, so no hidden content or directory listing is exposed alongside this finding.

4.7 F2: Anonymous SMB null session permitted on the domain controller

Affected system	DC01, `192.168.56.30` (Windows Server 2022, domain `michael.corp`)
Issue	The domain controller accepts an unauthenticated SMB session – no username or password is required to establish a session over port 445.
Severity	Medium
Evidence	See Figures 24–29. Confirmed by hand with `enum4linux-ng -A`, whose RPC Session Check reports that the server accepts authentication with an empty username and an empty password. The same null session also returned the NetBIOS domain name (`MICHAEL`), DNS domain (`michael.corp`), FQDN and domain SID (`S-1-5-21-978507740-1019232171-3849039492`) with no credentials supplied.
Why it matters	A null session is a standard reconnaissance foothold: with no credentials at all, an attacker can confirm domain membership and recover the domain SID, which is the starting point for RID-cycling attacks against user and group accounts. I also tested whether this extended to deeper enumeration (users, groups, shares, policies) via `enum4linux-ng`, `rpcclient` (SAMR and LSA calls, including `lookupnames`, `lookupsids`, `querydispinfo` and `enumdomusers`), an `nmap` SMB script scan (`smb-enum-shares`, `smb-enum-users`), and an anonymous LDAP bind; all were refused with `NT_STATUS_ACCESS_DENIED` or an LDAP operations error (Figures 26, 27, 28, 29), so the exposure here is limited to domain/SID disclosure rather than full directory enumeration. A follow-up `nmap` scan also confirmed SMB signing is enabled and required on the DC, which rules out SMB relay as a path arising from this finding.
Recommended fix	Set `RestrictAnonymous` to `2` under `HKLM\SYSTEM\CurrentControlSet\Control\Lsa` as a baseline hardening step (the Group Policy setting `Network access: Do not allow anonymous enumeration of SAM accounts and shares` sets the weaker value `1`). My verification showed that this alone did not stop the domain name and SID disclosure (Section 6), so further controls are needed, such as restricting the anonymous named pipes and shares the DC exposes. Test any change before using it on a production domain controller, because anonymous restrictions can affect legacy clients.

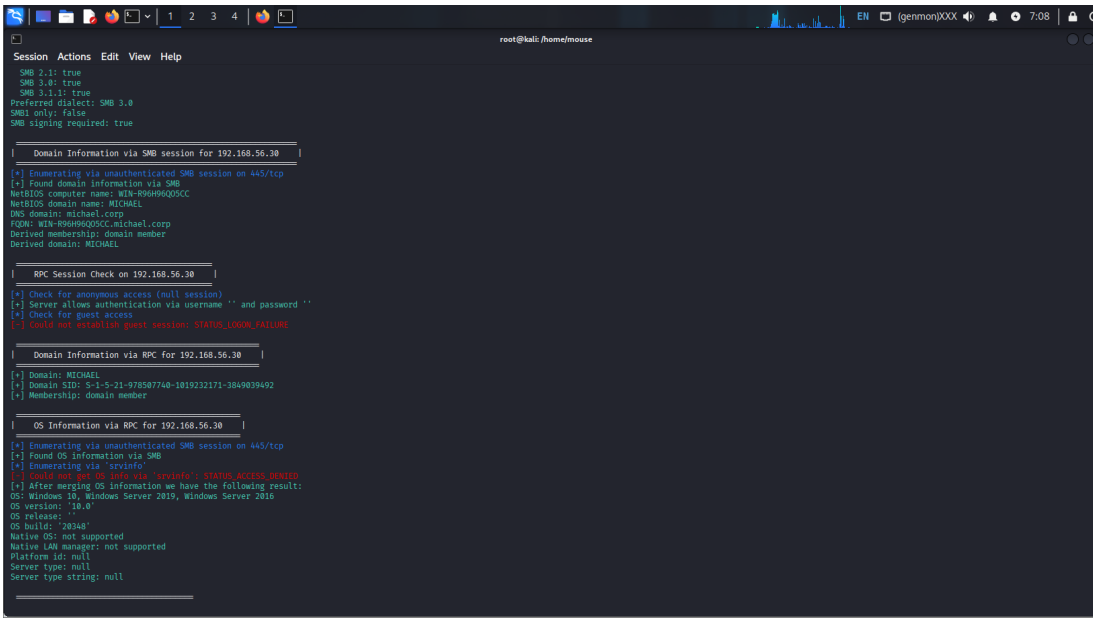


Figure 24: Evidence for finding F2 – anonymous SMB null session. I ran `enum4linux-ng -A 192.168.56.30` from Kali. The RPC Session Check section confirms that the server accepts a blank username and password.

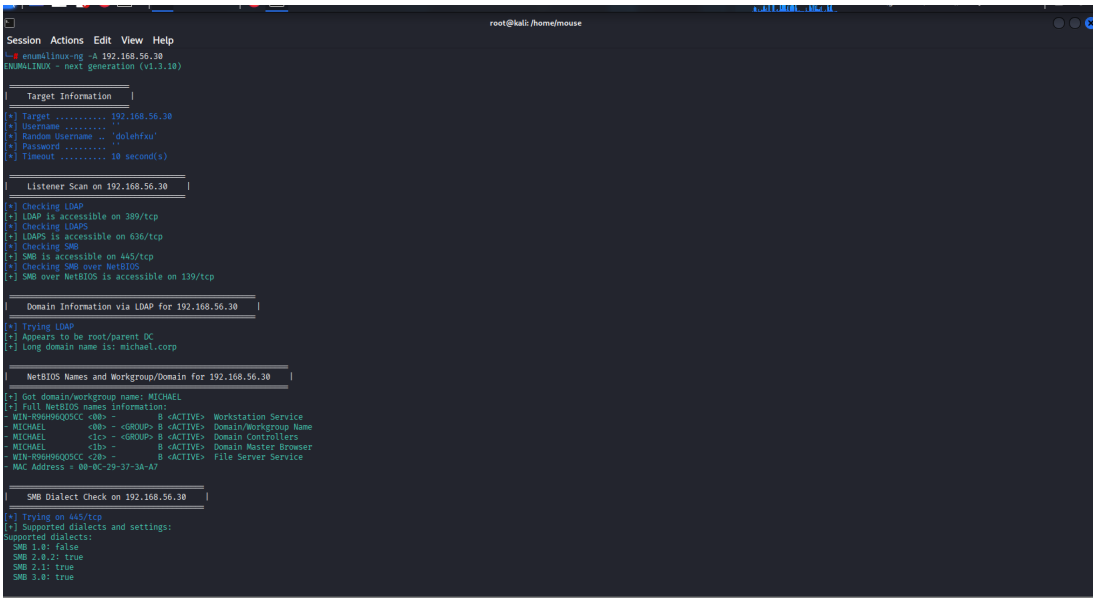


Figure 25: Domain information recovered through the null session. This part of the same `enum4linux-ng` output shows the domain and SID information recovered through the unauthenticated session: the NetBIOS name, DNS domain, FQDN and domain SID.

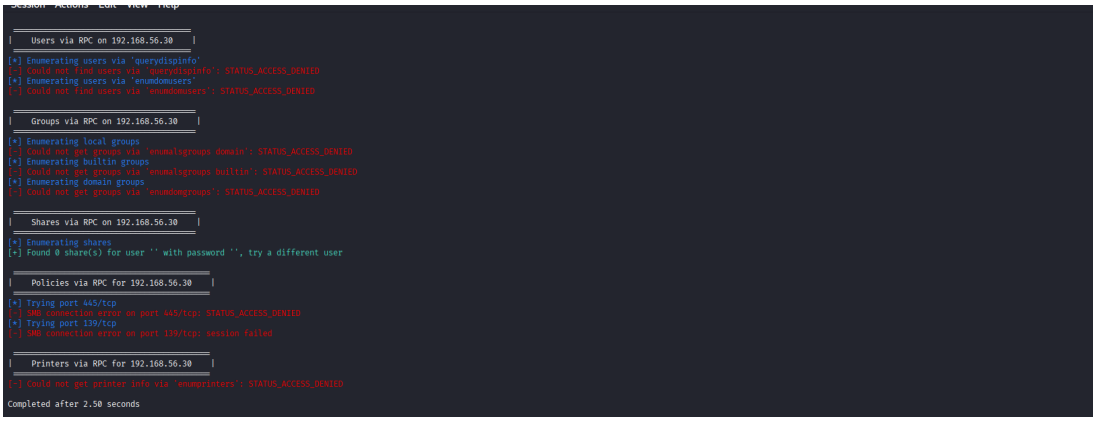


Figure 26: Deeper enumeration refused in enum4linux-ng. The users, groups, shares and policies sections of the same scan returned access denied, so the null session does not extend to full directory enumeration.

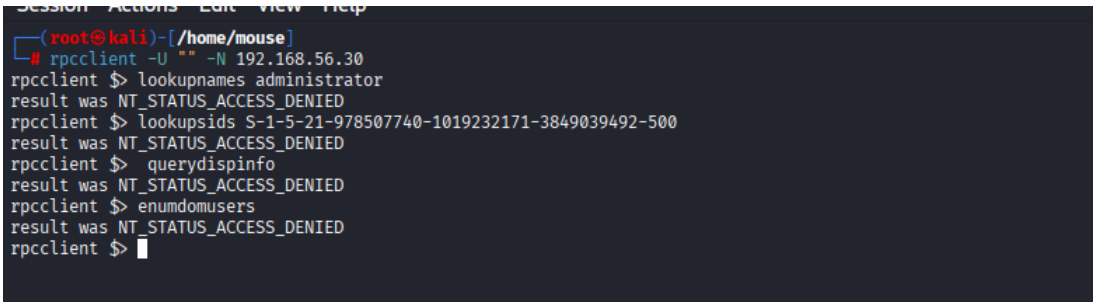


Figure 27: Deeper enumeration blocked despite the null session. I followed up with `rpcclient -U "" -N` and ran `lookupnames`, `lookupsids`, `querydispinfo` and `enumdomusers` by hand. All four returned `NT_STATUS_ACCESS_DENIED`, showing that SAMR- and LSA-based enumeration are blocked even though the null session itself is permitted (compare Figure 26).

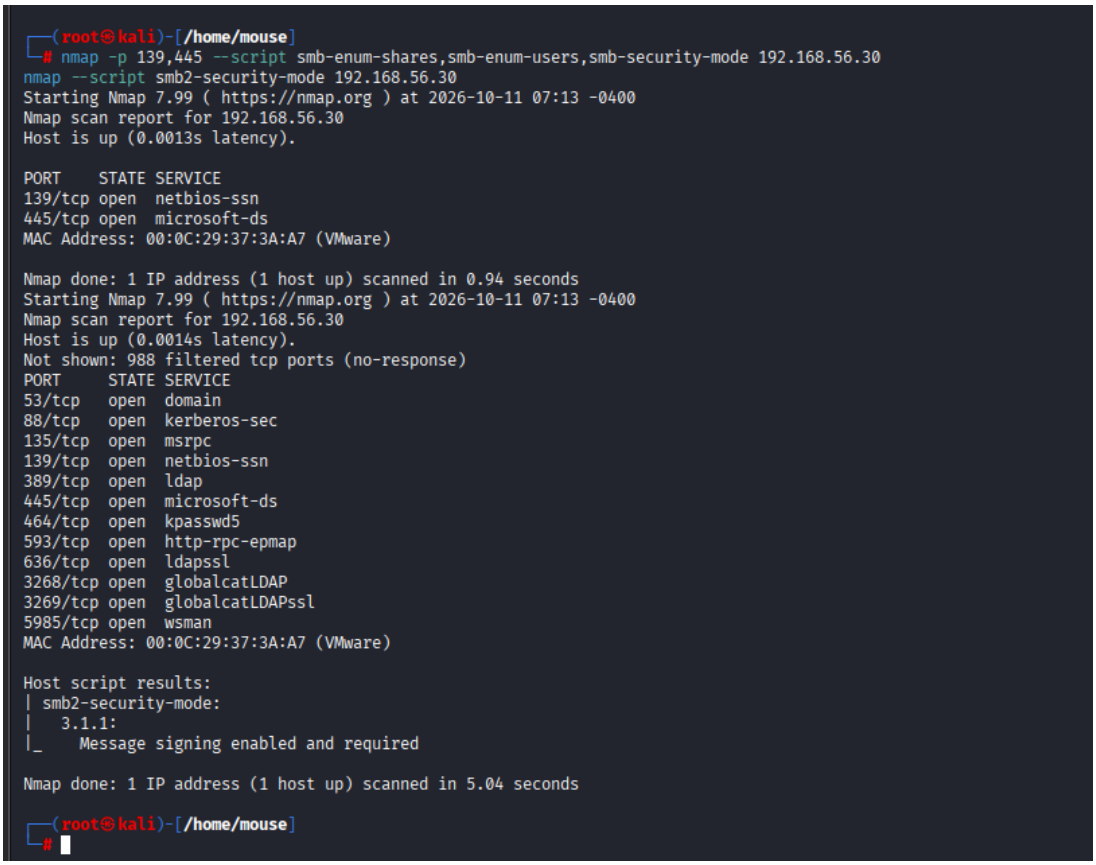


Figure 28: Nmap SMB script scan of the domain controller. I ran an `nmap` SMB script scan against DC01. It supports the results above and confirms that SMB signing is enabled and required, which rules out SMB relay as a path arising from this finding.

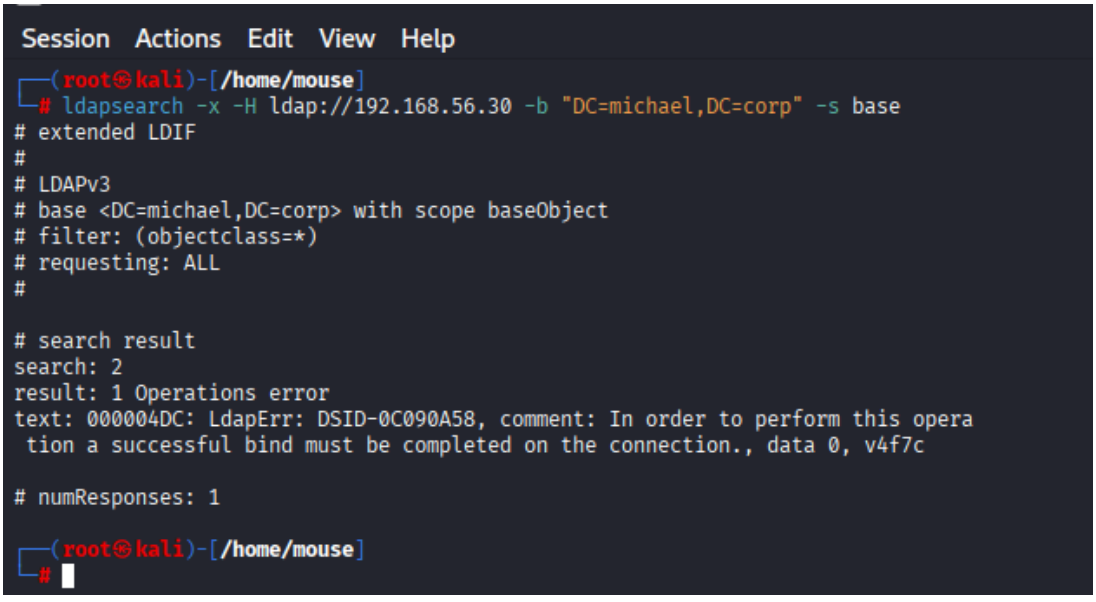


Figure 29: Anonymous LDAP bind refused. I tested an anonymous LDAP bind with `ldapsearch -x -H ldap://192.168.56.30 -b "DC=michael,DC=corp" -s base`. The server returned an Operations error requiring a successful bind first, so anonymous LDAP access is correctly restricted, in contrast to the SMB null session above.

4.8 F3: Host firewall on the Windows client blocks lateral-movement ports (informational)

Affected system	WIN-CLIENT, `192.168.56.20` (Windows 10 Home, not domain-joined)
Issue	Not a vulnerability: recorded as a positive observation. A targeted scan of the ports most commonly used for lateral movement on a Windows host (135, 139, 445, 3389, 5985) returned every port as `filtered` rather than open, even though the host responds to ICMP and is reachable on the lab network.
Severity	Informational
Evidence	See Figure 30. Confirmed with `nmap -sV -sC -Pn -p 135,139,445,3389,5985 -script smb-enum-shares,smb-enum-users,smb-security-mode,smb-os-discovery 192.168.56.20`, which returned all five ports filtered with no service detection possible.
Why it matters	This client has no domain membership and so receives no centrally enforced firewall policy (Section 7), yet the local Windows Defender Firewall defaults still block SMB, RDP and WinRM from the network. This meaningfully limits what an attacker who compromises another lab host could do against this machine, even without central management.
Recommended fix	No fix required. Recommend confirming this posture is preserved once the client is domain-joined or centrally managed, since it currently depends entirely on local defaults rather than an enforced policy.

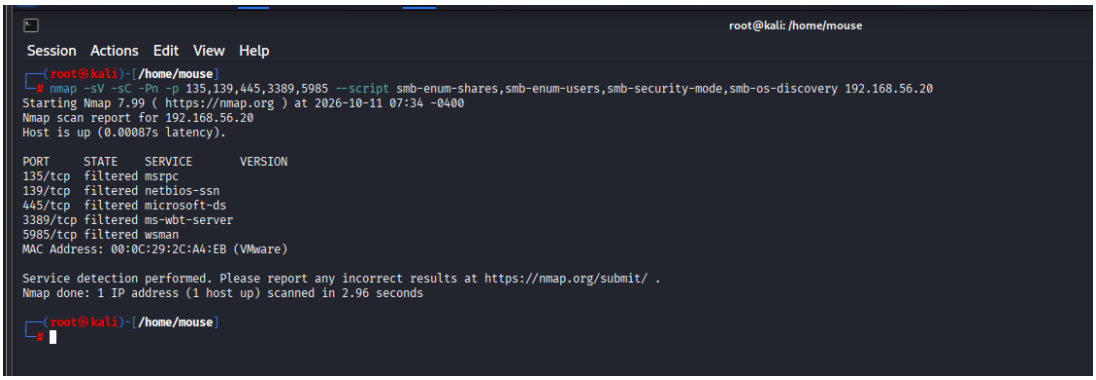


Figure 30: Evidence for finding F3 – WIN-CLIENT port filtering. I ran the scan above from Kali against WIN-CLIENT. All five targeted ports show as filtered, and no NSE scripts returned data, which is consistent with the host firewall silently dropping the probes rather than responding with a refusal.

4.9 Limitations of the Assessment

The assessment ran only from Kali on the same network segment as the targets, so it shows what an attacker with an internal foothold could see. It does not show how the systems would resist an attacker coming from outside. Everything outside the three lab systems on `192.168.56.0/24` was out of scope (Table 8). The Windows client was not domain-joined, so the assessment did not cover domain-member behaviour such as Group Policy or domain sign-in on a workstation. The testing was limited to the tools in Table 9, and only weaknesses that I could confirm by hand are reported.

5 Log Evidence and Detection Observations

What I did in this section. After the assessment, I looked at the lab from the defender's side by following these steps:

1. Noted the time range of my tests so that log entries could be matched to them.
2. Searched the Apache access log on the web server and the Windows Security log on the DC for entries from the Kali address `192.168.56.10`.
3. Matched each of my actions to log entries by time and source address (Table 12).
4. Checked the DC's audit policy and noted which activity left no useful trace.

5.1 Log Sources Reviewed

A defender only sees an attack through logs, so I reviewed the logs listed in Table 11 to find out which of my actions left a trace. I reviewed each log on the machine that produced it, because the lab has no central logging platform (Section 7).

Table 11: Log sources reviewed and why each was checked

Source	What it records	Why I reviewed it
Windows Security log on the DC	Sign-ins and authentication events, such as 4624 (successful logon) and 4625 (failed logon)	To see whether the anonymous sessions and enumeration attempts from my testing were recorded
Apache access log on the web server (`/var/log/apache2/access.log`)	Every request made to the website	To match web requests to my scans from `192.168.56.10`

5.2 Activity Correlation

Table 12 links each test I ran to the log entries it produced. I matched them by time and by the source address `192.168.56.10`. Apache records time in UTC (`+0000`). The DC's time zone is set to Pacific Standard Time, but its clock showed 12:46 on 11 October 2026 (Figure 35), which matches the host computer's clock (UTC) at that moment, so the entries can be compared directly with the Apache log and with the assessment window in Table 8 (10:51–11:38 UTC).

Table 12: My actions matched to the log entries they produced

Time (UTC)	My action	Log source	What the log showed
04:41:50 (first entry), 11:00:04 (last entry)	Nmap, WhatWeb, Nikto and Gobuster scans from Kali (`192.168.56.10`) against `WEB01`	Apache access log on `WEB01`	56,329 requests from `192.168.56.10`. Status codes: 55,913 × 404, 264 × 200, 91 × 400, 37 × 403, 13 × 501, 6 × 405, 4 × 523 and 1 × 417. User-Agents identify Gobuster (46,134 requests), WhatWeb (1 request) and Nikto's Shellshock test (122 requests). A further 9,840 requests carry a browser-style (Chrome) User-Agent that I could not attribute to a single tool.
11:06:43–11:06:45, then 11:19:31, 11:20:41 and 11:23:02	Null SMB session, `enum4linux-ng`, `rpcclient` and `ldapsearch` from Kali against `DC01`	Security log on `DC01`	16 Event 4624 (successful logon) entries for `ANONYMOUS LOGON`, all from `192.168.56.10`: 13 within the three seconds from 11:06:43 to 11:06:45, then one each at 11:19:31, 11:20:41 and 11:23:02. One Event 4625 (failed logon) at 11:06:44 for the account name `dolehfxu`, which is not one of the accounts I created.

The first Apache entry (04:41:50) is my earlier `curl -I` check that the homepage was served, which is before the assessment window. The scans themselves fall inside it.

Listing 10 shows the commands I used on the web server, and Listing 11 shows the query I used on the DC. Both filter on the Kali address.

Listing 10: Summarising web requests from the Kali IP

```
grep -c "192.168.56.10" /var/log/apache2/access.log
grep "192.168.56.10" /var/log/apache2/access.log | head -1
grep "192.168.56.10" /var/log/apache2/access.log | tail -1
grep "192.168.56.10" /var/log/apache2/access.log | awk '{print $9}' | sort | uniq -c | sort -rn
grep "192.168.56.10" /var/log/apache2/access.log | awk -F'"' '{print $6}' | sort | uniq -c |
    sort -rn
```

Listing 11: Searching the DC Security log for logon events from Kali

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624,4625} -MaxEvents 4000 |
  Where-Object { $_.Message -match '192\.168\.56\.10' } |
  Select-Object TimeCreated, Id
```



Figure 31: Web access log summary of requests from Kali. I ran the commands in Listing 10 on the web server to summarise the requests from `192.168.56.10`. The log holds 56,329 requests from Kali. The first is a `curl HEAD` request at 04:41:50 UTC, which is my earlier check that the homepage was served, and the last is at 11:00:04 UTC. Almost all requests returned 404 (55,913), which is typical of directory brute-forcing, and only 264 returned 200.

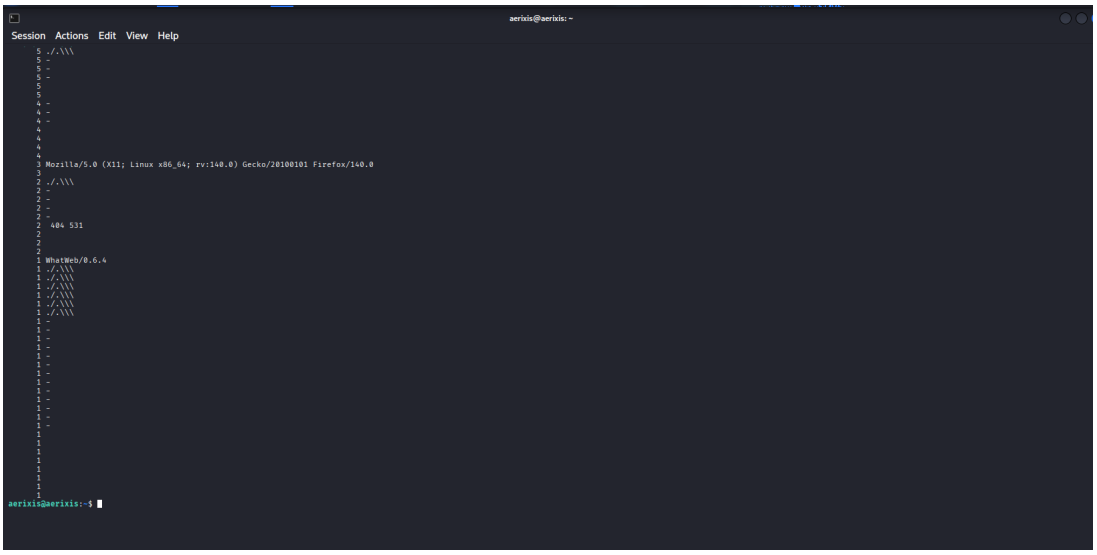


Figure 32: Web access log requests grouped by User-Agent. This is the rest of the User-Agent summary. Gobuster identifies itself as `gobuster/3.8.2` and produced 46,134 requests, and WhatWeb appears as `WhatWeb/0.6.4`. The 122 requests carrying a `() { :; }; echo ...cve-2014-6271` User-Agent are Nikto's test for the Shellshock vulnerability. The many blank and `-` lines are malformed or empty User-Agent fields from scanner probes.

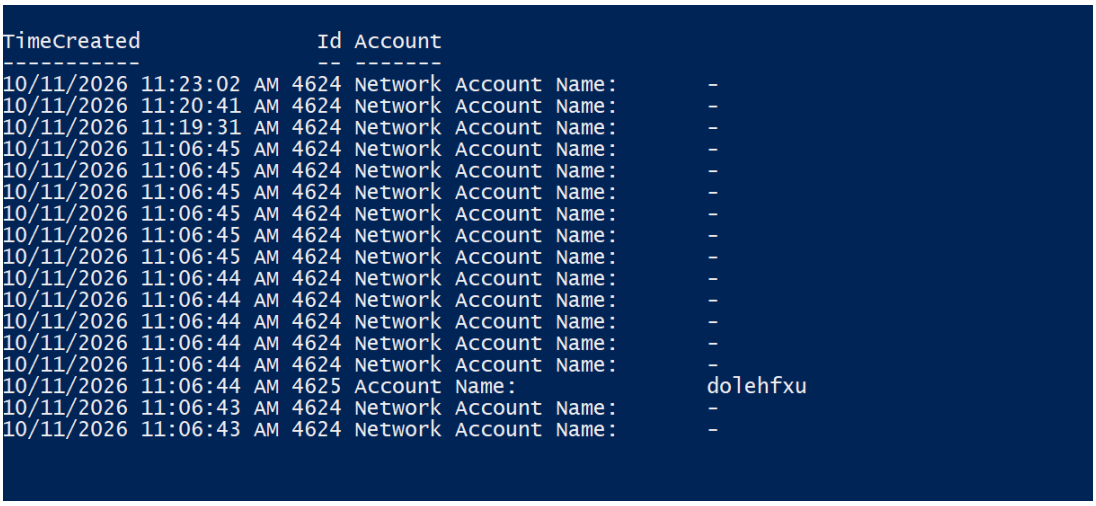


Figure 33: Windows Security log: logon events from Kali. I searched the Security log on DC01 for logon events (4624 and 4625) that mention `192.168.56.10`, using the query in Listing 11. It returned 17 events: sixteen 4624 entries and one 4625 at 11:06:44. The Account column prints the "Network Account Name" field, which is blank (`-`) on these events, so the account is confirmed in Figure 34. The 4625 shows the account name `dolehfxu`.

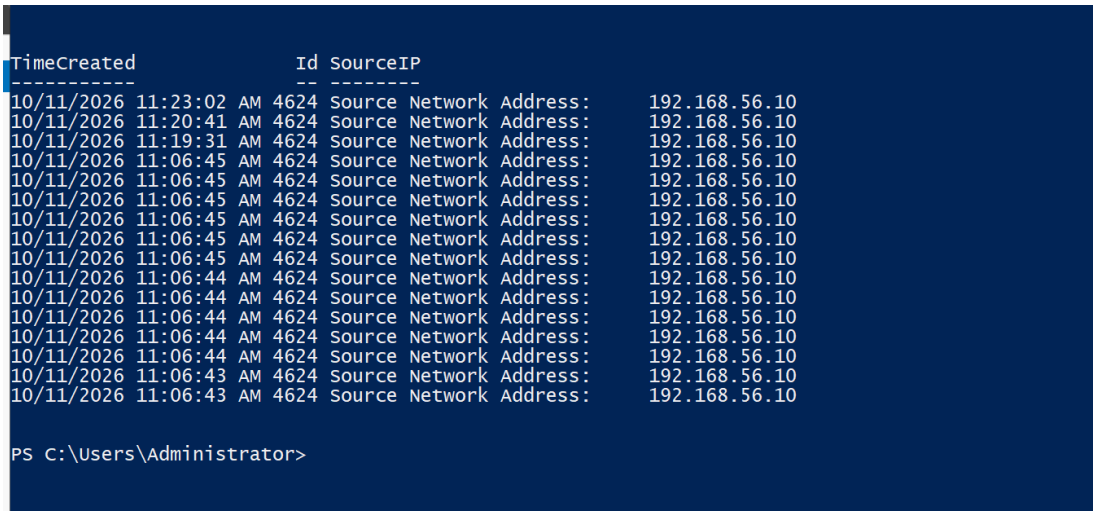


Figure 34: Anonymous logons and their source address. I filtered Event 4624 for entries whose message contains `ANONYMOUS LOGON` and printed the source network address. All 16 entries show `192.168.56.10`, and their times match the 4624 entries in Figure 33, so every successful logon from Kali in this search was anonymous.

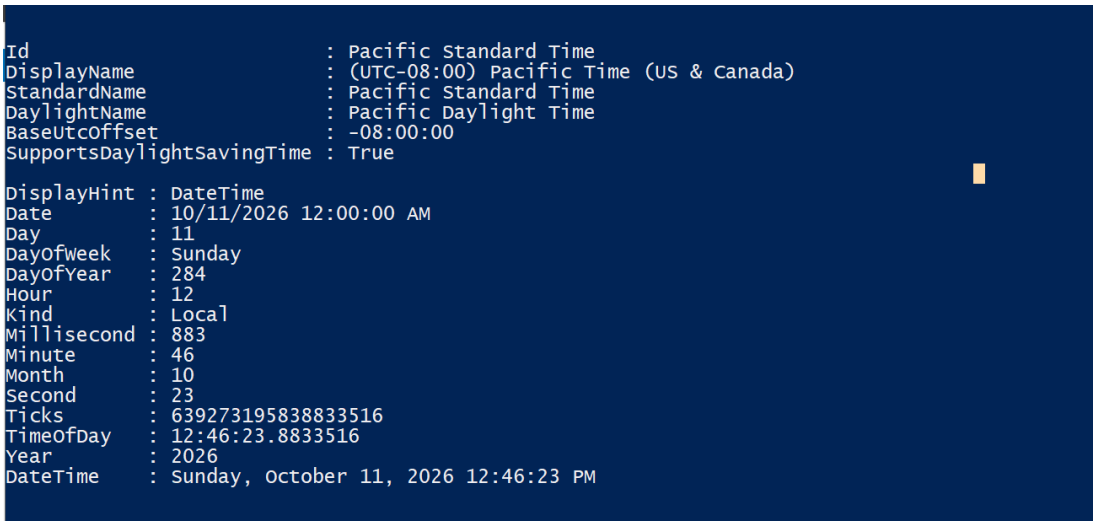


Figure 35: Time zone and clock on the domain controller. I checked the DC's time zone and clock so that the event times could be compared with the other logs. The zone is Pacific Standard Time (UTC−08:00), but the clock read 12:46 PM on 11 October 2026, which matches UTC at that moment.

5.3 What Was Not Visible

Logs were only available on each machine separately, so a defender would have to open every machine to piece an attack together. The DC did record my anonymous activity: Event 4624 shows the anonymous sessions and the Kali address, and Event 4625 shows one failed logon for an account name that does not exist in the domain. These entries are short and look alike, so they do not show which tool made them. They also show only that anonymous sessions were opened, not what was asked of them. In the events I searched (4624 and 4625), nothing shows the `rpcclient` requests for users or SIDs, the `NT_STATUS_ACCESS_DENIED` replies, or the refused anonymous LDAP bind. The audit policy (checked with `auditpol /get /category:"Logon/Logoff"`) had Logon set to Success and Failure and Special Logon set to Success, while Other Logon/Logoff Events, Group Membership and the IPsec subcategories were set to No Auditing. Compared with the web server, where Apache logged 56,329 requests from Kali, the DC left only a handful of events, so a defender would find the web scan far more easily than the enumeration of the DC.

6 Recommendations, Implemented Improvement and Verification

What I did in this section. I turned the findings into actions by following these steps:

1. Ranked the improvements by the risk they reduce (Table 13).
2. Chose one improvement to implement and recorded the exact change.
3. Ran the same check before and after the change and compared the results (Table 14).

6.1 Prioritised Recommendations

Table 13 ranks the improvements I recommend. The first three address my own findings and the others are standard controls that reduce the same kinds of risk.

Table 13: Prioritised recommendations, the risk each reduces and the finding it relates to

Priority	Recommendation	Risk addressed	Why it matters	Finding
1	Restrict anonymous SMB (`RestrictAnonymous` = 2)	Unauthenticated domain/SID disclosure	Hardens anonymous access, but verification showed that the domain and SID disclosure remained	F2
2	Add HTTP security headers to Apache	Reduced browser-side defence in depth	Limits the blast radius of any future web vulnerability on WEB01	F1
3	Monitor WIN-CLIENT firewall posture after domain join	Loss of local defaults once centrally managed	Ensures a GPO firewall policy does not silently relax what local defaults currently block	F3
4	Patch management and regular updates	Known vulnerabilities	Attackers often use flaws that already have a fix	–
5	Least privilege and a strong password policy	Account compromise	Limits what a stolen account can do and makes guessing harder	–
6	Host firewalls and network segmentation	Lateral movement and exposed services	Stops one compromised machine from reaching all the others	–
7	Backups with tested restores	Data loss and ransomware	Allows recovery if data is destroyed or encrypted	–
8	Central logging and alerting	Activity going unnoticed	Lets a defender see and react to attacks like my tests from one place	–

6.2 Implemented Improvement

Improvement: I set the registry value `RestrictAnonymous` to `2` on `DC01` (Listing 12) and restarted the server so that the setting took effect.
Reason: It directly addresses the confirmed finding F2, it is a single change that is easy to verify and to undo in the lab, and the starting state was already captured with `enum4linux-ng` (Figure 36).
Addresses finding: F2

Listing 12: Configuration change applied on DC01

```
reg add HKLM\SYSTEM\CurrentControlSet\Control\Lsa /v RestrictAnonymous /t REG_DWORD /d 2 /f
reg query HKLM\SYSTEM\CurrentControlSet\Control\Lsa /v RestrictAnonymous
shutdown /r /t 0
```

6.3 Verification

To test whether the improvement worked, I repeated the same check before and after the change (Table 14, Figures 36 to 39). I compared what the null session could still disclose, and not only whether the session connected, because restricting anonymous access can block the information without refusing the connection itself.

Table 14: Before-and-after comparison of the repeated check

Check repeated	What it tests	Before	After
`enum4linux-ng -A 192.168.56.30` (RPC Session Check and domain information)	Whether a null session (blank username and password) is accepted and what domain information it can still read	Accepted; domain name and SID disclosed (Figures 24, 25)	Null session still accepted. The RPC Session Check reports that the server allows authentication with an empty username and password, and the domain name, DNS domain, FQDN and domain SID are still returned (Figures 37, 38). Users, groups, shares, policies and printers are still refused (Figure 39), as before the change.

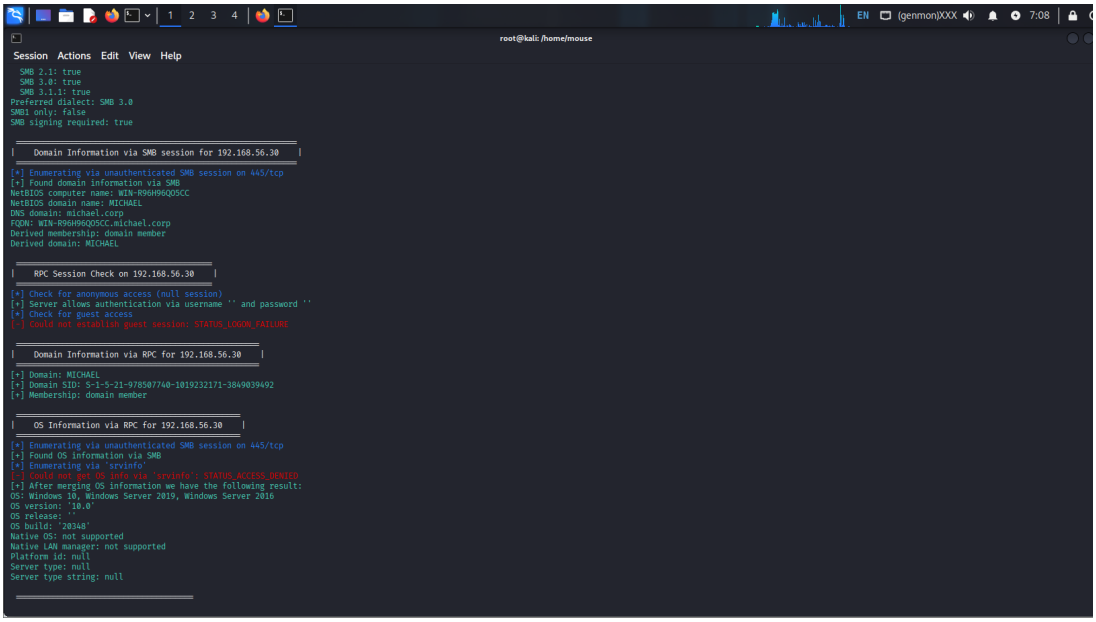


Figure 36: Check result before the improvement. I ran `enum4linux-ng -A 192.168.56.30` from Kali before changing anything, to record the starting state. The RPC Session Check shows that the null session was accepted, and the domain name and SID were disclosed (see also Figure 25).

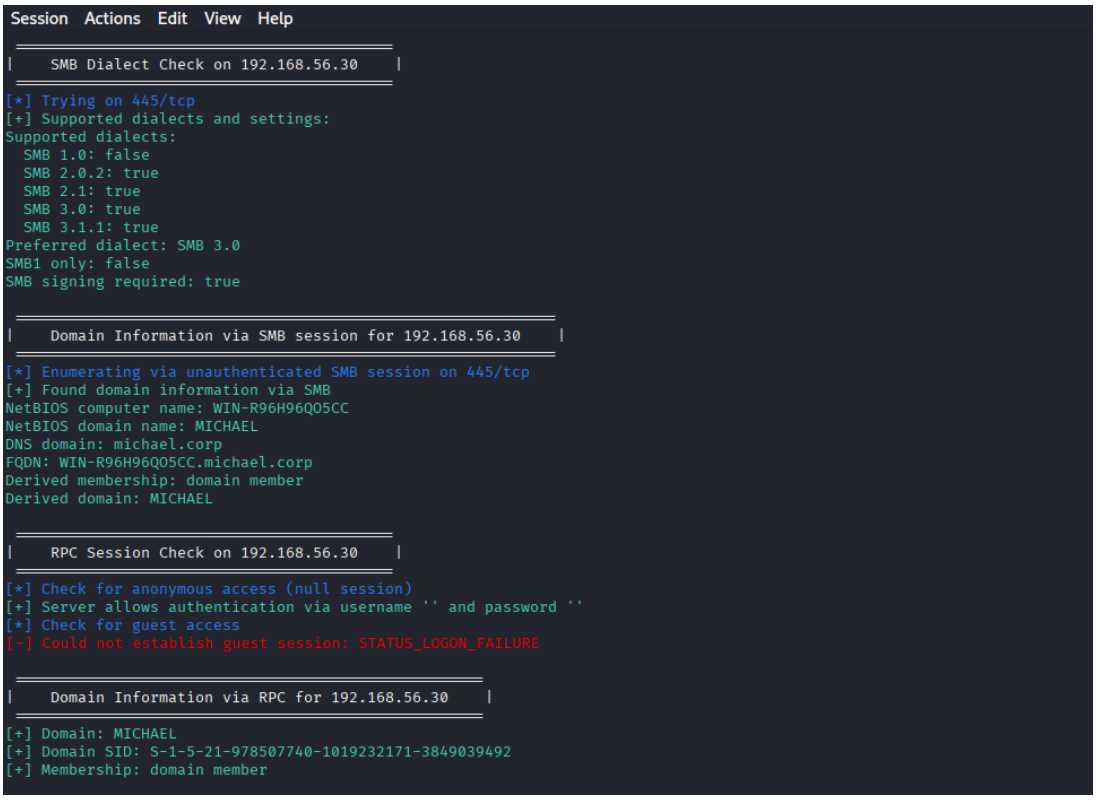


Figure 37: Check result after the improvement. I ran the same command, `enum4linux-ng -A 192.168.56.30`, after applying the change in Listing 12 and restarting the server. The RPC Session Check still reports that the server allows authentication with an empty username and password, and the domain name and domain SID are still returned through both the SMB session and RPC.



Figure 38: After the improvement: target and domain information. The first part of the repeat scan. LDAP, LDAPS and SMB are still reachable, the long domain name `michael.corp` is still returned through LDAP, and the NetBIOS names and domain controller roles are still listed.

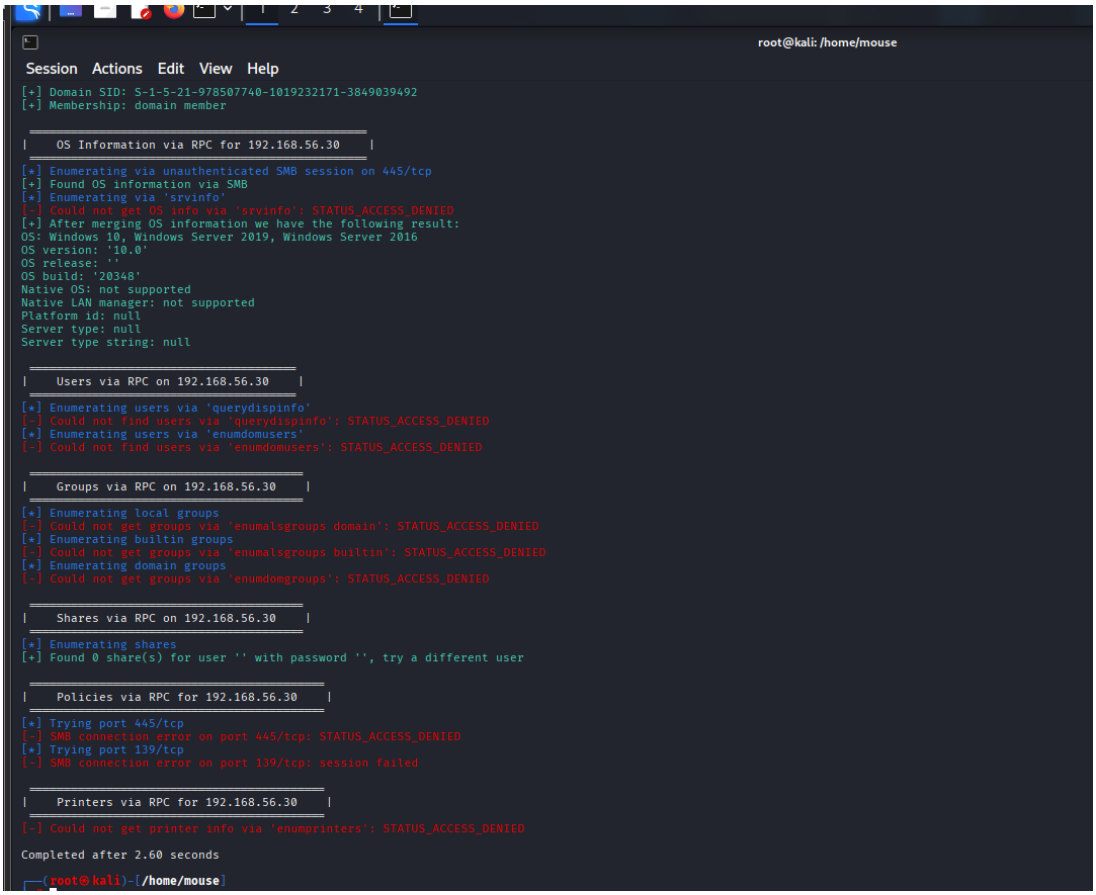


Figure 39: After the improvement: deeper enumeration still refused. The rest of the repeat scan. Enumeration of users, groups, policies and printers still returns `STATUS_ACCESS_DENIED`, and no shares are listed for the empty user. This matches the result before the change (Figure 26).

Result. Setting `RestrictAnonymous` to 2 did not change what the null session could read in this test. The session was still accepted and the domain name and SID were still disclosed, so the improvement did not close finding F2. The deeper enumeration that was already refused (users, groups, shares, policies) stayed refused, so the change caused no visible regression, but it also added no visible protection against this disclosure. The gap that remains is the domain and SID information returned through the unauthenticated SMB session and anonymous RPC. Closing it would need a different control, for example restricting which anonymous named pipes and shares the DC exposes, and I did not test that. This shows why an improvement has to be verified: the setting was applied correctly, but it did not do what the finding assumed.

7 Challenges, Limitations and Lessons Learned

What I did in this section. I reviewed the whole project and recorded (1) the problems I met and how I handled them, (2) the limits of the lab and the assessment, and (3) what I learned.

7.1 Challenges

Domain join not possible on Windows 10 Home. I tried to join the client to `michael.corp`, but the **Domain** option in the Computer Name/Domain Changes window was greyed out, and the window states that a computer running this edition of Windows 10 cannot join a domain (Figure 9). Before that, `nslookup` and `nltest /dsgetdc:michael.corp` had already shown

that the client could resolve the domain and reach the DC at `192.168.56.30`, so DNS and connectivity were not the cause. Settings > About showed that the client runs Windows 10 Home, which does not support domain join. A Windows 10 Pro (or Enterprise or Education) edition is needed to join a domain, and I did not have one for this client. As a result, **I could not join the client to the domain** and it stayed in the workgroup `WORKGROUP`. I did not rebuild the client because the other assessment targets (the DC and the web server) were all reachable, but it means the lab does not show a domain user signing in on a workstation. In future I would upgrade the client to Windows 10 Pro or rebuild it from a Windows 10 Enterprise Evaluation ISO, then run the join command in Listing 4.

DNS validation warning from netsh. When I set the DNS server on the client with `netsh`, it reported that the DNS server was incorrect or did not exist. `ipconfig /all` showed the setting had been applied, and a ping and an `nslookup` against the DC both worked, so the warning was only netsh's validation check. I repeated the command with `validate=no`.

No IPv4 address on the web server after installation. After the first boot of `WEB01`, `ip a` showed only IPv6 link-local addresses on both adapters, and `ping google.com` failed with "Temporary failure in name resolution". Ubuntu Server had not configured IPv4 on either adapter. I fixed it by writing a netplan file with DHCP on the NAT adapter (`ens33`) and a static address on the host-only adapter (`ens37`), as described in Section 3. Netplan is the standard tool on Ubuntu Server, so I used it instead of installing NetworkManager as on Kali.

Limited host hardware. The laptop that hosts the lab does not have enough resources to run a monitoring platform such as Wazuh alongside four virtual machines, so I did not deploy one. I reviewed the Windows and Linux logs directly on each machine instead.

First scan on WIN-CLIENT came back as host down. When I pointed Nmap at `192.168.56.20` without `-Pn`, it reported the host as down, even though the machine answered ping earlier in the project. Nmap's default host-discovery probe is not plain ICMP; it can be dropped by the same Windows Firewall defaults that later turned out to be the basis of F3. Adding `-Pn` to skip host discovery and scan the ports directly resolved this and produced the filtered-port result used as evidence for F3.

Different clocks on different machines. Each machine reports time in its own way: Apache logs in UTC, Kali's local time is UTC-4, and the DC is set to Pacific Standard Time while its clock reads the same as UTC. Before matching log entries to my tests I checked the DC's time zone and clock (Figure 35). My first query on the DC used Kali's local time and returned no events, and only after I widened the search and checked the clocks did the anonymous logons appear. A central log platform with synchronised time would remove this problem.

The first fix did not close the finding. Setting `RestrictAnonymous` to `2` did not stop the domain name and SID disclosure (Section 6). I reported this result as it was and did not adjust the finding to match my expectation.

7.2 Limitations

The lab is small: four virtual machines on one host and a single network segment. All accounts were created by me, and no real users generate sign-ins or web traffic, so the logs do not reflect real-world user behaviour. The Windows client runs Windows 10 Home and could not be joined to the domain, so the lab does not include a domain-joined workstation and I could not test domain sign-in from a client. The assessment was run only from inside the network (Kali on the same segment), so it does not cover attacks that start from the internet. Because the network is isolated, I could not test controls that depend on an internet-facing perimeter. The lab also has no central logging or alerting, so I could not show how a defender would detect my activity from a single dashboard. Central logging remains a recommendation in Table 13. Finally, I tested

only one improvement, and it did not fully close its finding, so the remaining recommendations are untested.

7.3 What I Learned

Technically, this project showed me the gap between a scanner result and a confirmed finding. Nikto and enum4linux-ng both pointed at weaknesses quickly, but the useful part of the work was going a layer deeper each time: following up the SMB null session with `rpcclient` calls made by hand, and testing an anonymous LDAP bind as a second path into the same question, rather than stopping at the first tool's output. That habit also produced an unexpected result – the SAMR and LSA enumeration attempts were blocked even though the null session itself was permitted – which taught me that a single "vulnerable" label rarely covers a whole service; DC01 was simultaneously misconfigured in one respect and correctly hardened in another, and the report is more accurate for showing both. Verifying my own fix showed that a correctly applied setting did not close the finding, which taught me to test a fix and not assume it works.

I also learned to be more careful about what a scan result actually proves. The first Nmap scan against WIN-CLIENT came back as "host down" even though I knew from earlier connectivity tests that the machine was reachable, and tracking that down to Nmap's host-discovery probe (rather than plain ICMP) being silently dropped was a reminder that a negative result needs the same scrutiny as a positive one before it goes in a report. The log review taught the same lesson: my first query on the DC found nothing, and an empty result turned out to come from a wrong time window and not from missing logs.

From the defender's side, the review showed how uneven detection can be. The web scan left 56,329 Apache entries that identify the tools by name, while the enumeration of the domain controller left only a few short logon events. That difference is the strongest argument I found for central logging and for auditing more than logon events.

On the professional side, writing this report alongside doing the testing – rather than testing first and writing afterwards – made it much easier to keep the evidence honest: every finding here is backed by a screenshot I can point to, and I did not fill in a result that I had not observed. That discipline matters more in a real assessment than in a lab exercise, since a client report that overstates what was tested is worse than one that is incomplete but honest.

8 References and Supporting Material

8.1 Acknowledgements

- AI assistance: I used an AI assistant (Claude, by Anthropic) to help structure and format this LaTeX report, to suggest wording for figure captions and tables, to explain how to create users and groups in Active Directory Users and Computers, how to set the client's IP and DNS settings, how to configure the web server's network adapters and how to check the Apache version, to help diagnose the failed domain join, to generate the HTML, CSS and JavaScript content of the WEB01 homepage used as the assessment target, to help interpret and write up the results of the Nikto, Gobuster, enum4linux-ng, rpcclient, nmap and ldapsearch commands run during the assessment, and to suggest the commands for searching the Apache and Windows logs and help interpret their output. I built and configured the lab myself, ran every command and took every screenshot, and checked the technical statements in this report against my own systems.
- Video tutorial: I used a YouTube tutorial [4] to learn the steps for joining a Windows client to an Active Directory domain.

8.2 References

References

[1] NIST, *SP 800-115: Technical Guide to Information Security Testing and Assessment*, 2008.

[2] Nmap Project, *Nmap Reference Guide*, `https://nmap.org/book/man.html`.

[3] Microsoft, *Active Directory Domain Services overview*, Microsoft Learn.

[4] YouTube video tutorial on joining a Windows client to an Active Directory domain, `https://youtu.be/0gdYu6bw0O0`, accessed 11 October 2026.

A Appendix: Commands and Code Reference

This appendix collects every command, script and configuration file referenced in this report in one place, grouped by stage, so they can be re-run or checked without searching through the full text.

A.1 A.1 Domain Controller Setup

Listing 13: Creating a domain user and adding it to a group in PowerShell (equivalent to the graphical steps used in Section 3.2.1)

```
New-ADUser -Name "Helpdesk Admin" -SamAccountName "hadmin" ‘
  -Path "OU=labUsers,DC=michael,DC=corp" ‘
  -AccountPassword (Read-Host -AsSecureString) -Enabled $true
Add-ADGroupMember -Identity "Staff" -Members "hadmin"
```

Listing 14: Command-line verification of accounts and groups on DC01

```
net user
net groups
```

A.2 A.2 Windows Client Configuration

Listing 15: Static IP, DNS server and ping firewall rule on WIN-CLIENT (run as administrator)

```
netsh interface ip set address name="Ethernet1" static 192.168.56.20 255.255.255.0
netsh interface ip set dns name="Ethernet1" static 192.168.56.30 validate=no
netsh advfirewall firewall add rule name="Allow ICMPv4 In (Lab)" protocol=icmpv4:8,any dir=in
    action=allow remoteip=192.168.56.0/24
```

Listing 16: Pre-domain-join checks run on WIN-CLIENT

```
ipconfig /all
ping 192.168.56.30
nslookup michael.corp
nltest /dsgetdc:michael.corp
```

Listing 17: Domain join command (not completed – blocked by Windows 10 Home edition, Section 3.3.2)

```
Add-Computer -DomainName michael.corp -Credential MICHAEL\Administrator -Restart
```

A.3 A.3 Linux Web Server (WEB01) Setup

Listing 18: Netplan configuration – /etc/netplan/50-cloud-init.yaml

```
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: true
    ens37:
      dhcp4: false
      addresses:
        - 192.168.56.40/24
```

Listing 19: Applying the netplan configuration

```
sudo netplan apply
```

Listing 20: Installing and enabling Apache

```
sudo apt update && sudo apt install -y apache2
sudo systemctl enable --now apache2
```

Listing 21: Checking the Ubuntu release and Apache version

```
lsb_release -a
apache2 -v
```

Listing 22: Deploying the homepage and confirming Apache serves it

```
mv /home/aerixis/index.html /var/www/html/index.html
chown root:root /var/www/html/index.html
chmod 644 /var/www/html/index.html
curl -I http://192.168.56.40
```

Listing 23: Connectivity check from WEB01 to the Windows client

```
ping -c 4 192.168.56.20
```

A.4 A.4 Kali Linux Setup

Listing 24: Persistent static IP on eth1 with NetworkManager

```
nmcli con add type ethernet ifname eth1 con-name eth1-static \
  ipv4.method manual ipv4.addresses 192.168.56.10/24
nmcli con up eth1-static
```

Listing 25: Two-way connectivity test between Kali and WIN-CLIENT

```
ping -c 4 192.168.56.20   # run from Kali
ping -c 4 192.168.56.10   # run from WIN-CLIENT
```

A.5 A.5 Web Server Assessment (WEB01)

Listing 26: WhatWeb fingerprinting

```
whatweb -v http://192.168.56.40/
```

Listing 27: Manual header check

```
curl -I http://192.168.56.40
```

Listing 28: Gobuster directory brute-force for hidden content and backup files

```
gobuster dir -u http://192.168.56.40 -w /usr/share/wordlists/dirb/common.txt -x php,bak,txt,zip
```

Listing 29: Nikto web server scan

```
nikto -h http://192.168.56.40
```

A.6 A.6 Domain Controller Assessment (DC01)

Listing 30: SMB/AD service discovery and script scan

```
nmap -p 139,445 --script smb-enum-shares,smb-enum-users,smb-security-mode 192.168.56.30
nmap --script smb2-security-mode 192.168.56.30
```

Listing 31: enum4linux-ng full enumeration (null session, domain/SID disclosure, RPC/LDAP checks)

```
enum4linux-ng -A 192.168.56.30
```

Listing 32: Anonymous LDAP bind test

```
ldapsearch -x -H ldap://192.168.56.30 -b "DC=michael,DC=corp" -s base
```

Listing 33: rpcclient manual null-session enumeration (SAMR/LSA calls)

```
rpcclient -U "" -N 192.168.56.30

# inside the rpcclient session:
lookupnames administrator
lookupsids S-1-5-21-978507740-1019232171-3849039492-500
querydispinfo
enumdomusers
```

Listing 34: Fix verification – repeat the null-session check after applying RestrictAnonymous = 2

```
# On DC01, apply the fix:
reg add HKLM\SYSTEM\CurrentControlSet\Control\Lsa /v RestrictAnonymous /t REG_DWORD /d 2 /f

# From Kali, repeat the original check:
enum4linux-ng -A 192.168.56.30
```

A.7 A.7 Windows Client Assessment (WIN-CLIENT)

Listing 35: Targeted port scan of common lateral-movement services

```
nmap -sV -sC -Pn -p 135,139,445,3389,5985 --script smb-enum-shares,smb-enum-users,smb-security-
    mode,smb-os-discovery 192.168.56.20
```

A.8 A.8 Full Lab Discovery Scan

Listing 36: Full-range service discovery scan of all three in-scope hosts

```
nmap -sV -sC -p- -oN scans/lab_full.txt 192.168.56.20 192.168.56.30 192.168.56.40
```

A.9 A.9 Log Correlation

Listing 37: Extracting assessment-related web requests from the Apache access log

```
grep "192.168.56.10" /var/log/apache2/access.log | head -20
```